

Operating System

Harshit Batra

Quiz

Live Poll

About this Course

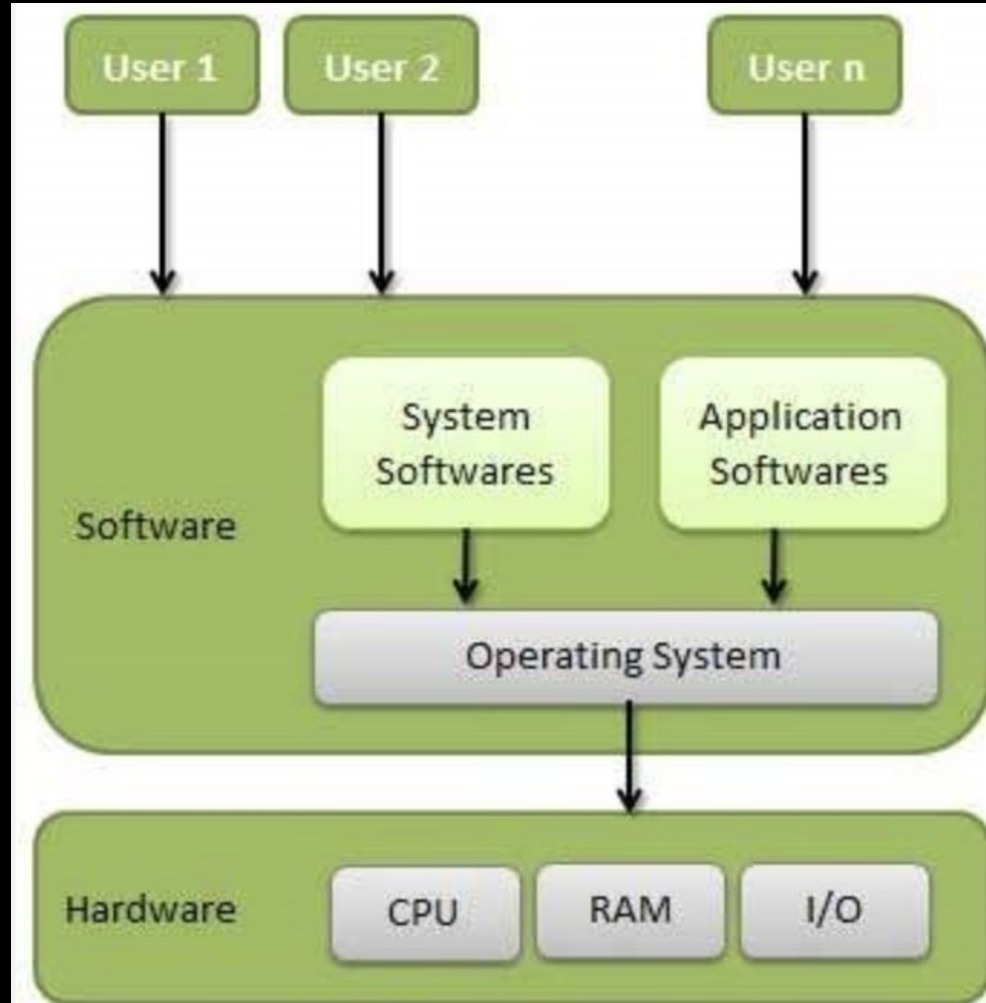
- OS Programming: to use OS
- OS Implementation: to write OS
- C Language
- Linux Commands

Core Components

- Syllabus Overview
 - *Three easy pieces*
 - Virtualization
 - Concurrency
 - Persistence

Why do we need OS?

- Why do we need Virtualization?



OS Fundamental Concepts

Virtualization

- CPU Virtualization
- Memory Virtualization

Concurrency

- threads

Persistence

- File System

OS: Three easy pieces

Virtualization (Process and Memory)

- 1.OS gives every process a feeling that it has own CPU
2. Every process feels that its enough memory

Concurrency

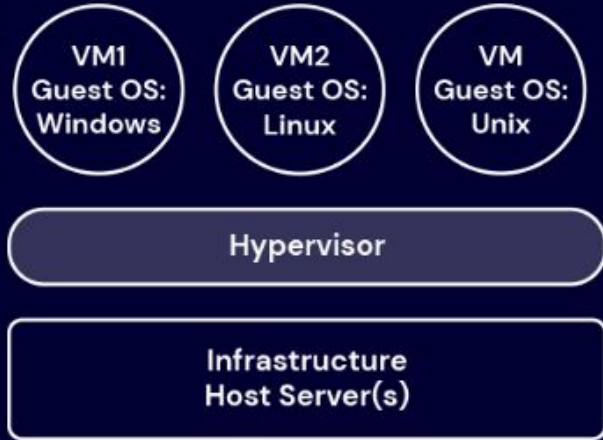
1. Multiple process can run at the same time without resulting in problems
2. OS Provides mechanisms to make them work together

Persistence

1. Disk is an I/O device. It needs to be managed and handled.
2. OS handles interactions with the disk and performs storage management

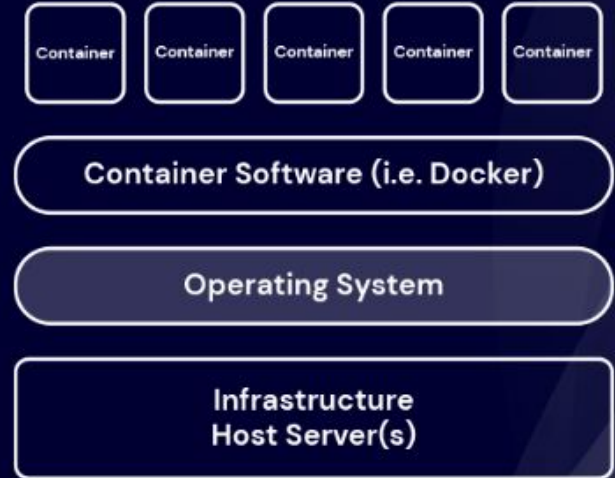
Virtual Machines vs Containers

Virtual Machines



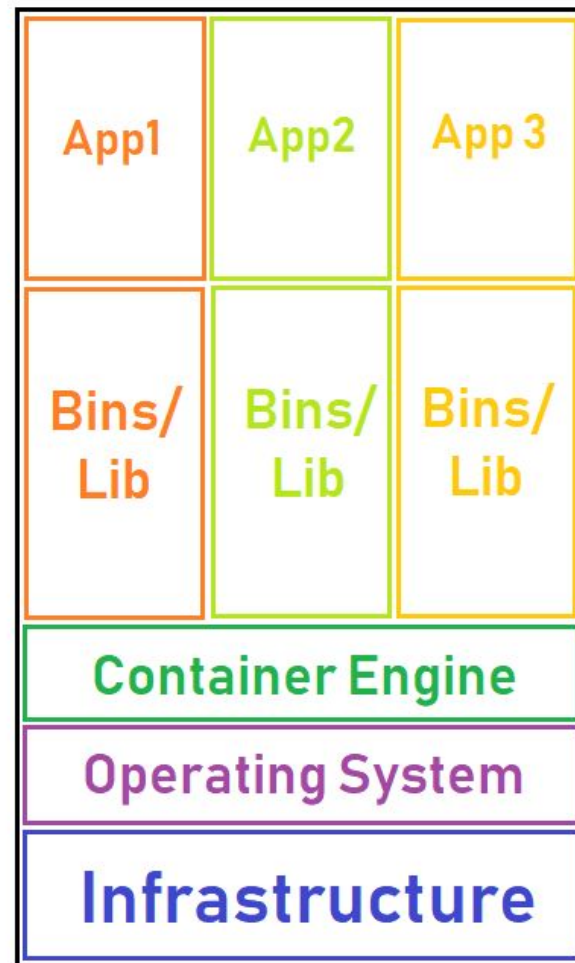
WHAT'S
the
DIFF?

Containers





Virtualization



Containerization

Abstraction vs Virtualization

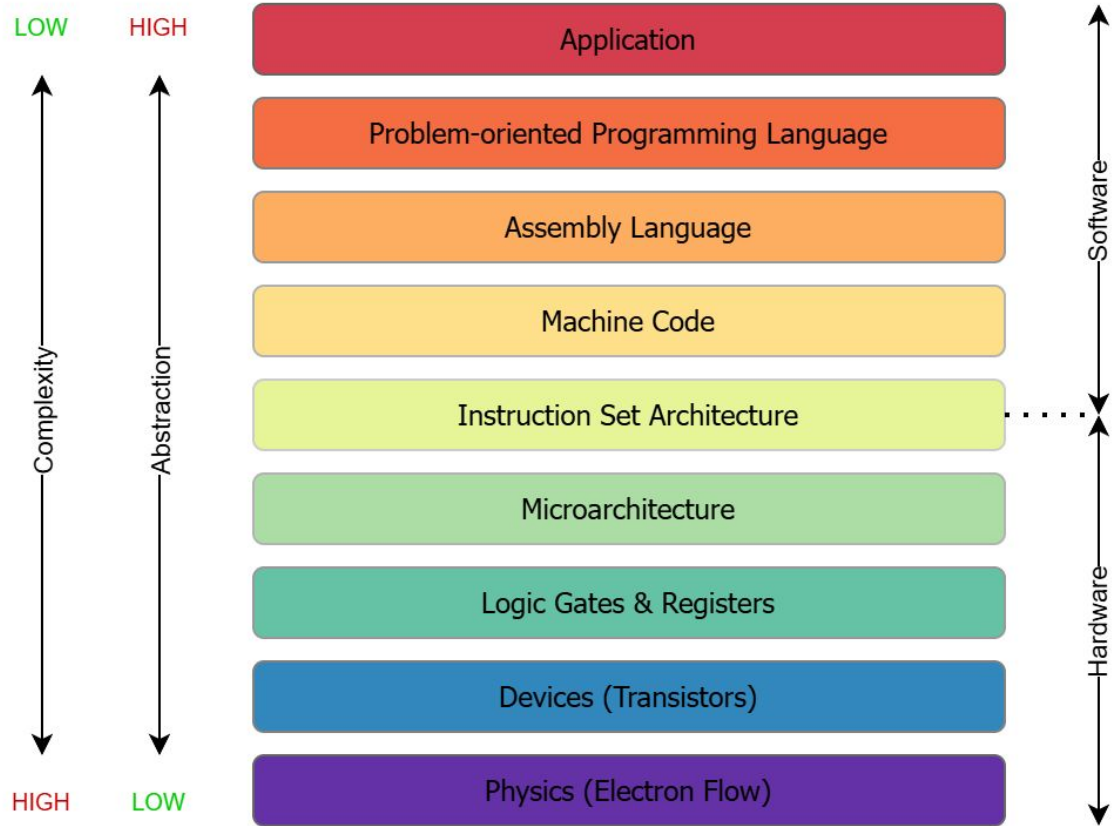
- Abstraction hides complexity. Virtualization share resources
- Virtualization creates the illusion of a resource.
- Virtualization itself is built using abstractions.
- Abstraction simplifies the interaction with resources. Virtualization controls resources or what you appear to own

Abstraction = Complexity

Virtualization = Resources

Abstraction

Layers of Abstraction

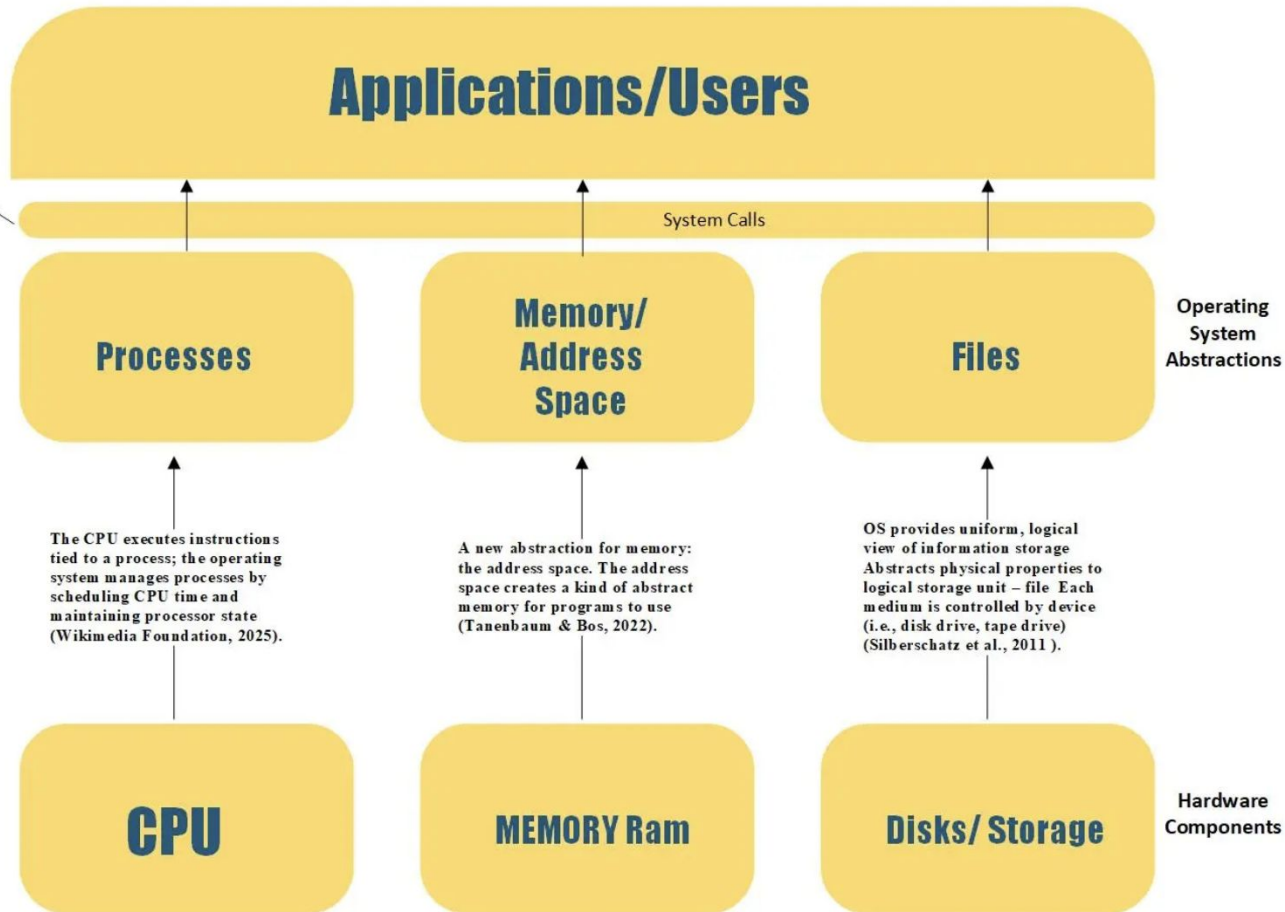


Abstraction

Physical Hardware	What hardware actually provides	OS Abstraction	What the programmer sees
CPU	Executes machine instructions on a limited number of cores	Process / Thread	<code>main()</code> , threads, processes
RAM	Physical memory addresses and memory cells	Virtual Memory / Address Space	Variables, pointers, <code>malloc()</code>
SSD / HDD	Blocks/sectors, controllers, physical storage	File / Directory	<code>open()</code> , <code>read()</code> , <code>write()</code>
Network Card (NIC)	Sends/receives Ethernet/IP packets	Socket	<code>socket()</code> , <code>send()</code> , <code>recv()</code>
Keyboard	Electrical/device events generated by key presses	Input events / Device interface	<code>stdin</code> , keyboard events
Display / GPU	Pixels, framebuffer, GPU commands	Display / graphics API	Window, framebuffer, graphics APIs
Timer / Clock	Hardware timer interrupts/counters	Timer / Time API	<code>sleep()</code> , timers, clocks
USB devices	USB packets, device protocols	Device files / drivers	Read/write device interface

Diagram: The Operating System as an Abstraction

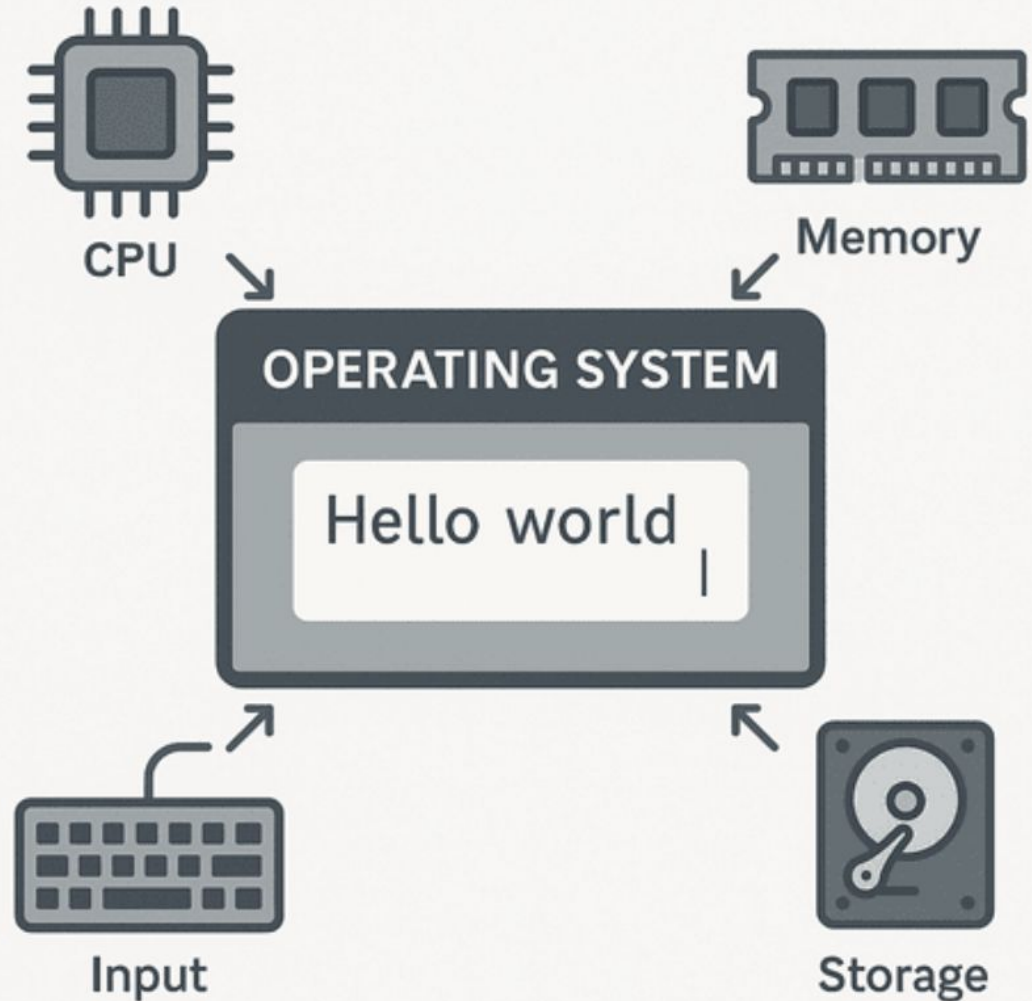
Applications need to access system resources such as memory, files, and networks. System calls provide a standardized interface for applications to request these resources from the OS (Prepbytes, 2023).

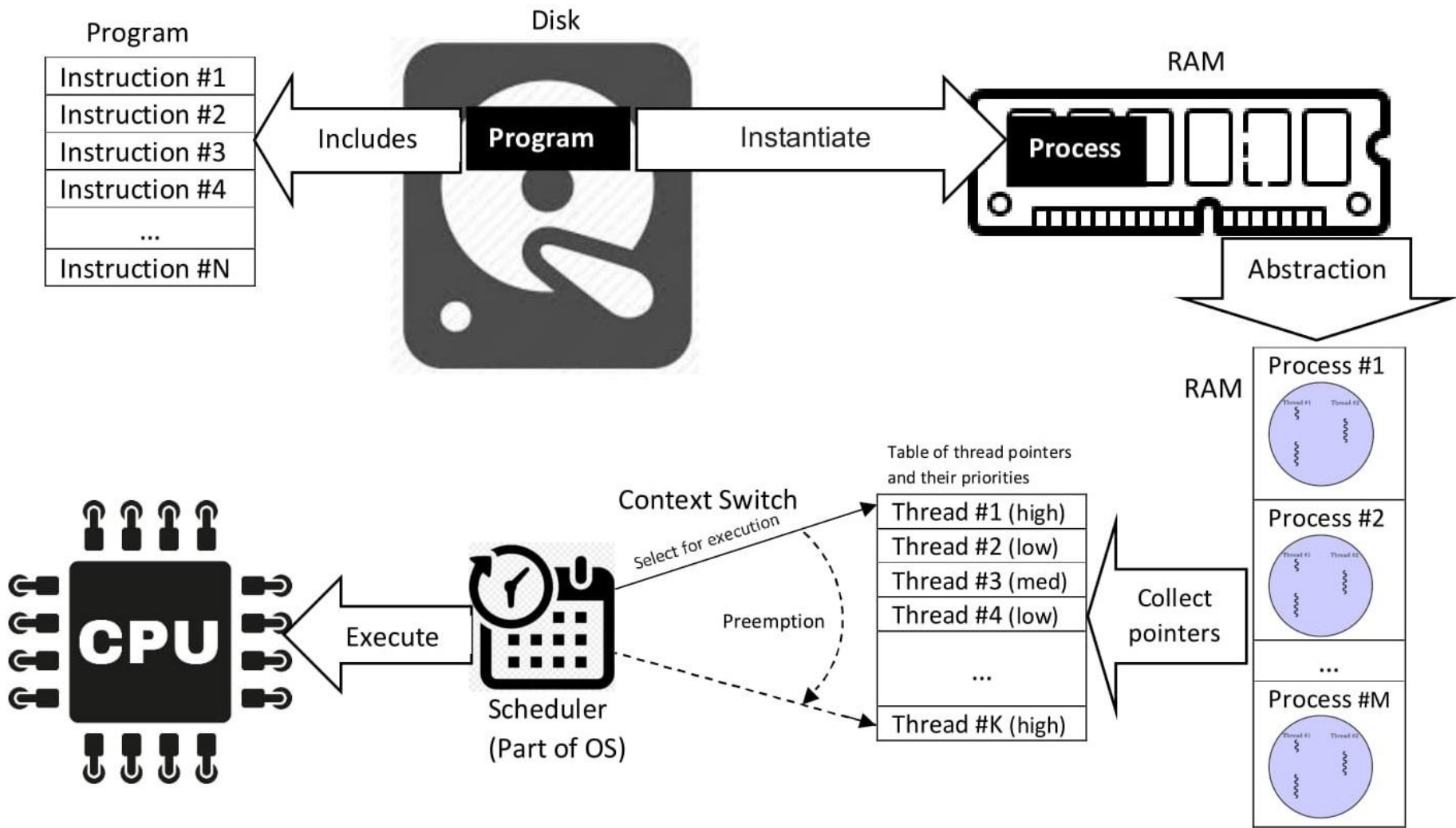


OS: Resource Manager

Resources

- CPU
- Memory or RAM
- I/O: Storage or HDD/SSD, etc





GM-NAA I/O,
produced by
General
Motors for its
IBM 704

1956



Apple II
released

1977



MS-DOS is
released by
Microsoft

1981



Linux is
released
by Linus
Torvalds

1991



Windows 95
is released

1995



android

Android is
released
(based on
a Linux
kernel)

2008



OPENSIFT

OpenShift
released
by Red
Hat

2011

Timeline of Operating Systems

1960s

IBM develops a series of OSs for its 360 series. Multics is developed and abandoned but UNIX is developed as a consequence.

1970s

Unix becomes popular in academic circles and spawns many versions



1980s

The home computer revolution



1990s

Windows dominates the laptop and desktop market



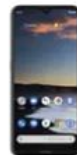
2000s

Unix and then Linux dominate the Supercomputer Market



2010s

Smart phones become ubiquitous after the iPhone release in 2007



C Program Compilation

samp.c

©2004 HowStuffWorks

```
#include <stdio.h>

int main()
{
    printf("Hello!\n");
    return 0;
}
```

samp.c is your C Program. You type samp.c into a text file using a standard text editor. It is human-readable.



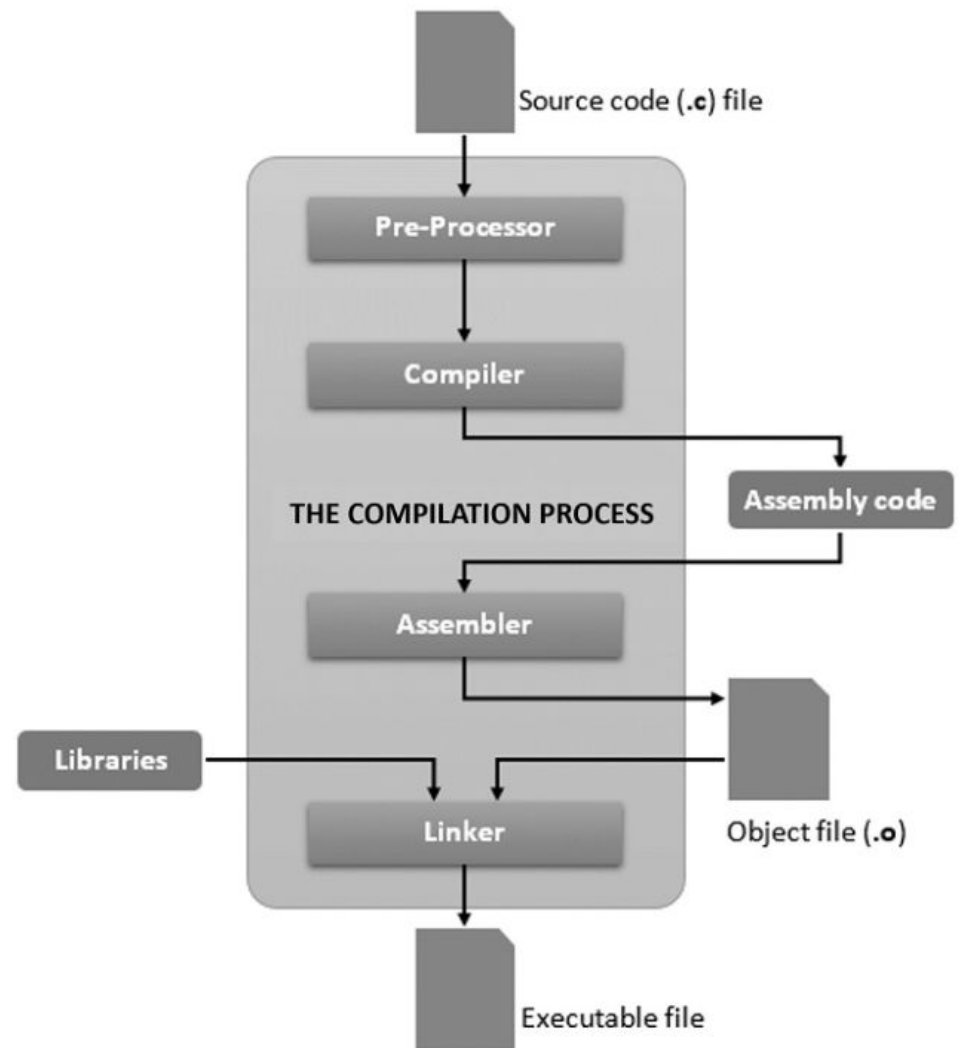
You type:
gcc samp.c -o samp.exe
to compile samp.c into
samp.exe using the gcc
compiler.

samp.exe

```
10110101001011010
10100100100100010
10101001010101110
01010010010110101
11101010100111001
10101001010111110
10101101101001001
10101000111101011
```

The C compiler takes samp.c as input and turns it into a machine-readable executable. The computer "runs" or "executes" this executable.

C Program Compilation



Environment setup

Isolation

- **Container Isolation:** Each container has its own isolated environment, allowing processes to function independently within their own namespace as if they were the only ones running on the system.
 - Each container runs in its own namespace and cannot access or interact with the resources of other containers unless specifically allowed.
 - **Namespace isolation** establishes logical boundaries for each container's processes, networks, and resources to enable the safe and efficient execution of multiple containers on the same system.
-
- Container = isolate not virtualize

Namespace Isolation



Storage

Device Mapper

Btrfs

Aufs

Namespaces

PID

MNT

IPC

UTS

NET

Networking

veth

bridge

iptables

Cgroups

cpu

cpuset

memory

device

Security

Capability

SELinux

seccomp

Namespace

```
vscode → /workspaces/OS $ lsns
```

	NS	TYPE	NPROCS	PID	USER	COMMAND
	4026531834	time	21	117	vscode	/bin/sh
	4026531837	user	21	117	vscode	/bin/sh
	4026532556	mnt	21	117	vscode	/bin/sh
	4026532557	uts	21	117	vscode	/bin/sh
	4026532558	ipc	21	117	vscode	/bin/sh
	4026532559	pid	21	117	vscode	/bin/sh
	4026532560	cgroup	21	117	vscode	/bin/sh
	4026532561	net	21	117	vscode	/bin/sh

Userspace vs Namespace

- Userspace is where your application lives. A **memory area** where normal applications, user programs, and shells run.
- Namespace is a kernel mechanism that controls the application's view of the system. It **isolates global system resources** for a group of processes.
- Userspace **protects the operating system** core from crashing via bad app code.
- Namespaces **hide system resources** from other processes to create containers
- Userspace: **devcontainers/base:noble**

Docker VM Managers

Docker Desktop for Mac currently supports different VM managers:

- Docker VMM — Apple Silicon
- Apple Virtualization framework
- QEMU — legacy on Apple Silicon
- HyperKit — legacy on Intel Macs

Summary

1. VM

Virtualizes hardware

Provides a Linux kernel

Example: Docker Desktop's Linux VM

2. Container

Isolates processes

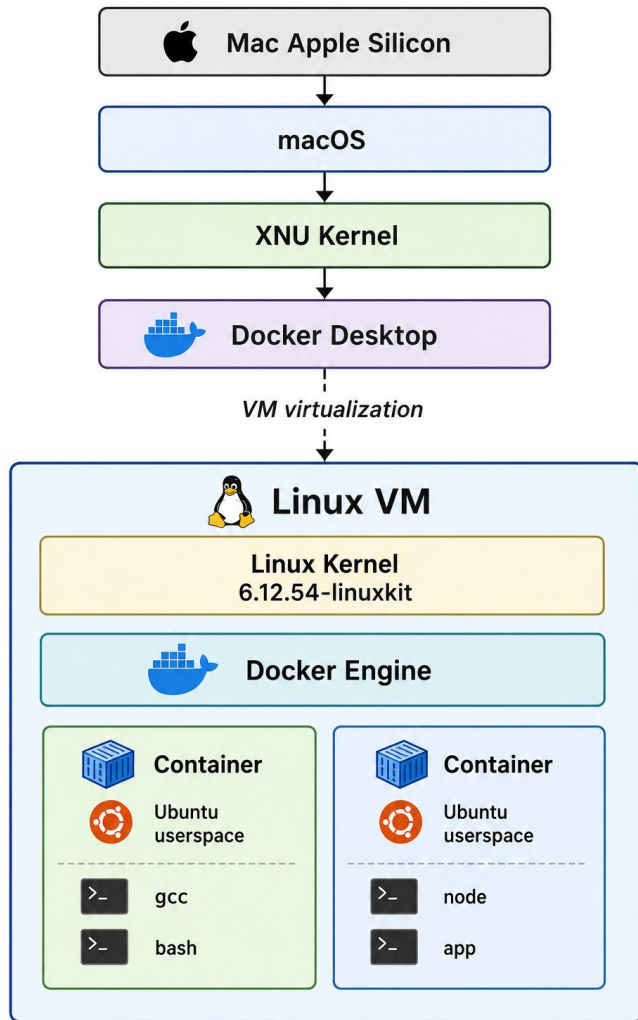
Shares the Linux kernel

Uses namespaces + cgroups + capabilities

3. Userspace

Where applications execute

bash, gcc, Node, your C program, libc, etc.

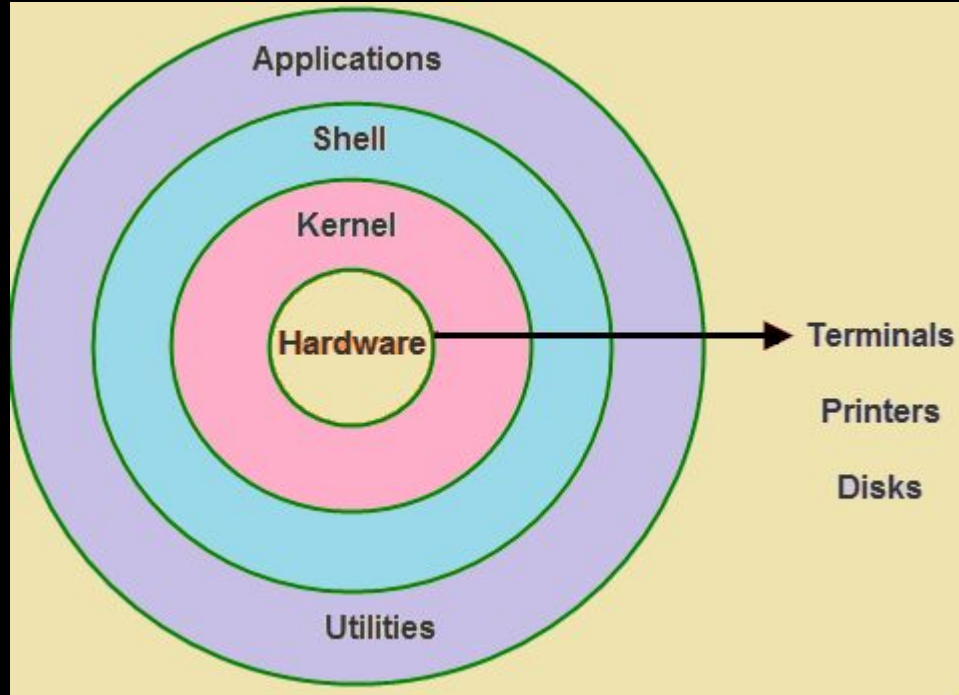


System Calls

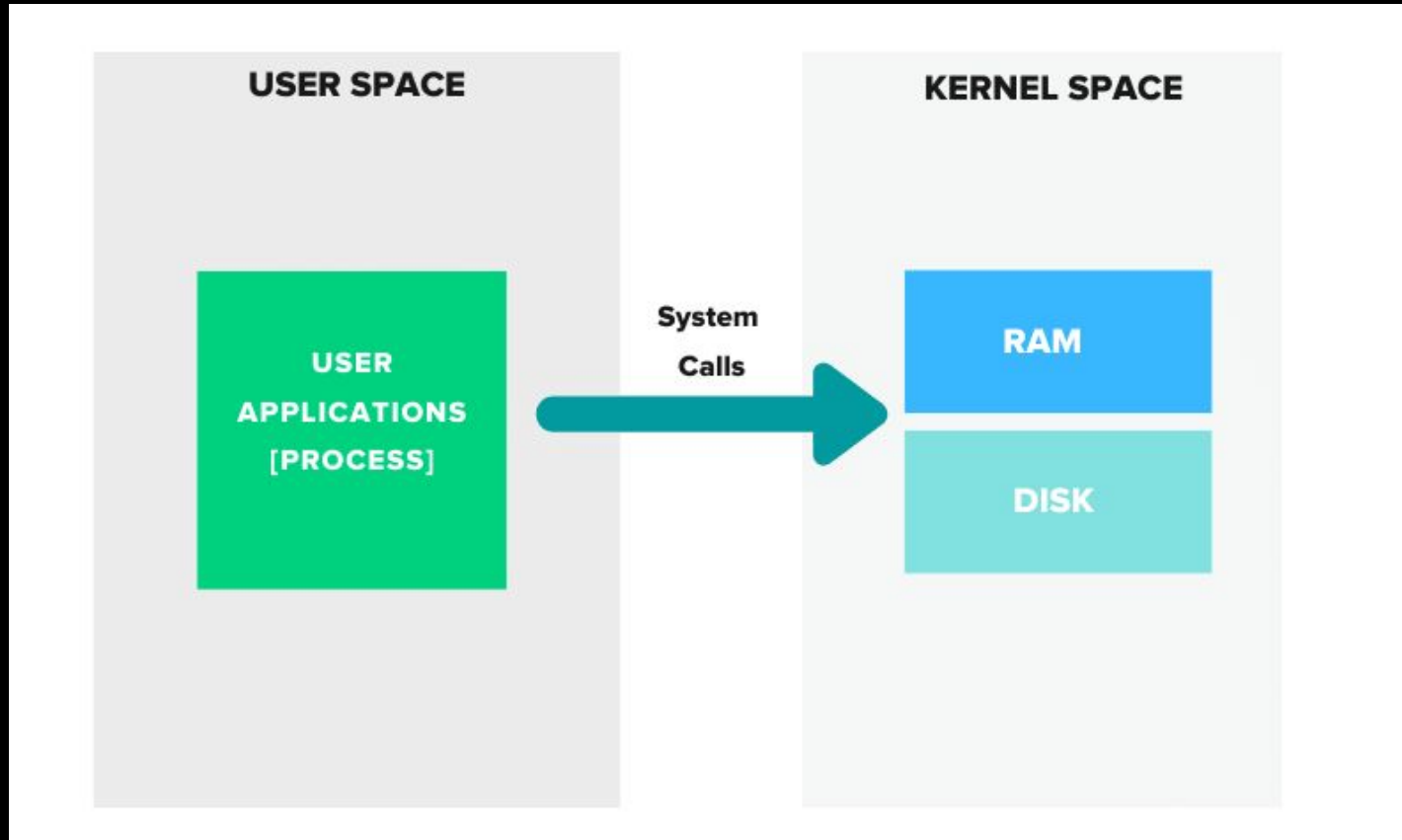
System Calls

- System calls are the **programmatic interface** that allows a user application to request services directly from the operating system kernel.
 - They act as a secure bridge, letting software **interact safely with physical hardware** and core system resources without risking system stability.
 - To protect the system, modern **operating systems divide execution into two spaces**:
 - **User Mode**: *Restricted environment where standard applications run.*
 - **Kernel Mode**: *Privileged environment with unrestricted access to system hardware.*
 - When a program needs a resource (like reading a file), it executes a specific trap or software interrupt. This **switches the CPU from User Mode to Kernel Mode**, executes the requested service safely inside the kernel, and then switches back
-
- **Interrupt** is a signal sent to a computer's processor by hardware or software demanding immediate attention

Shell & Kernel

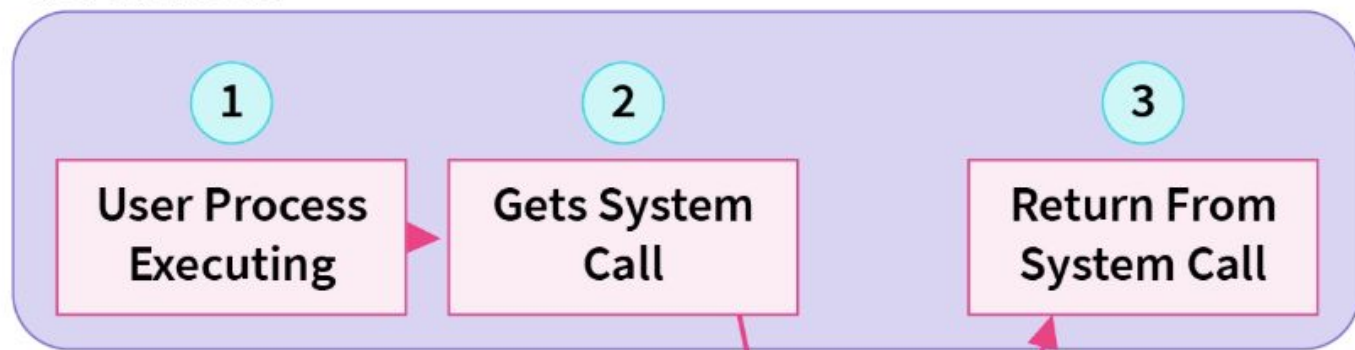


User Mode & Kernel Mode

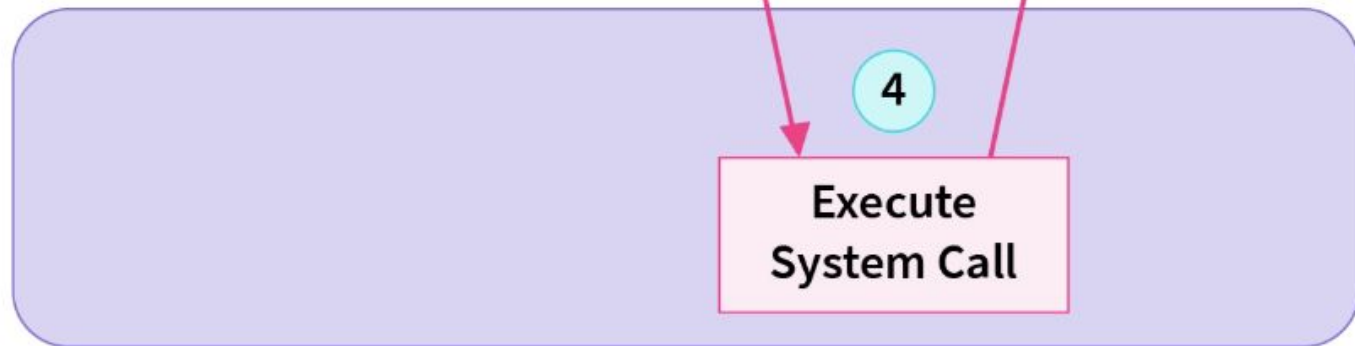


WORKING OF A SYSTEM CALL

USER MODE

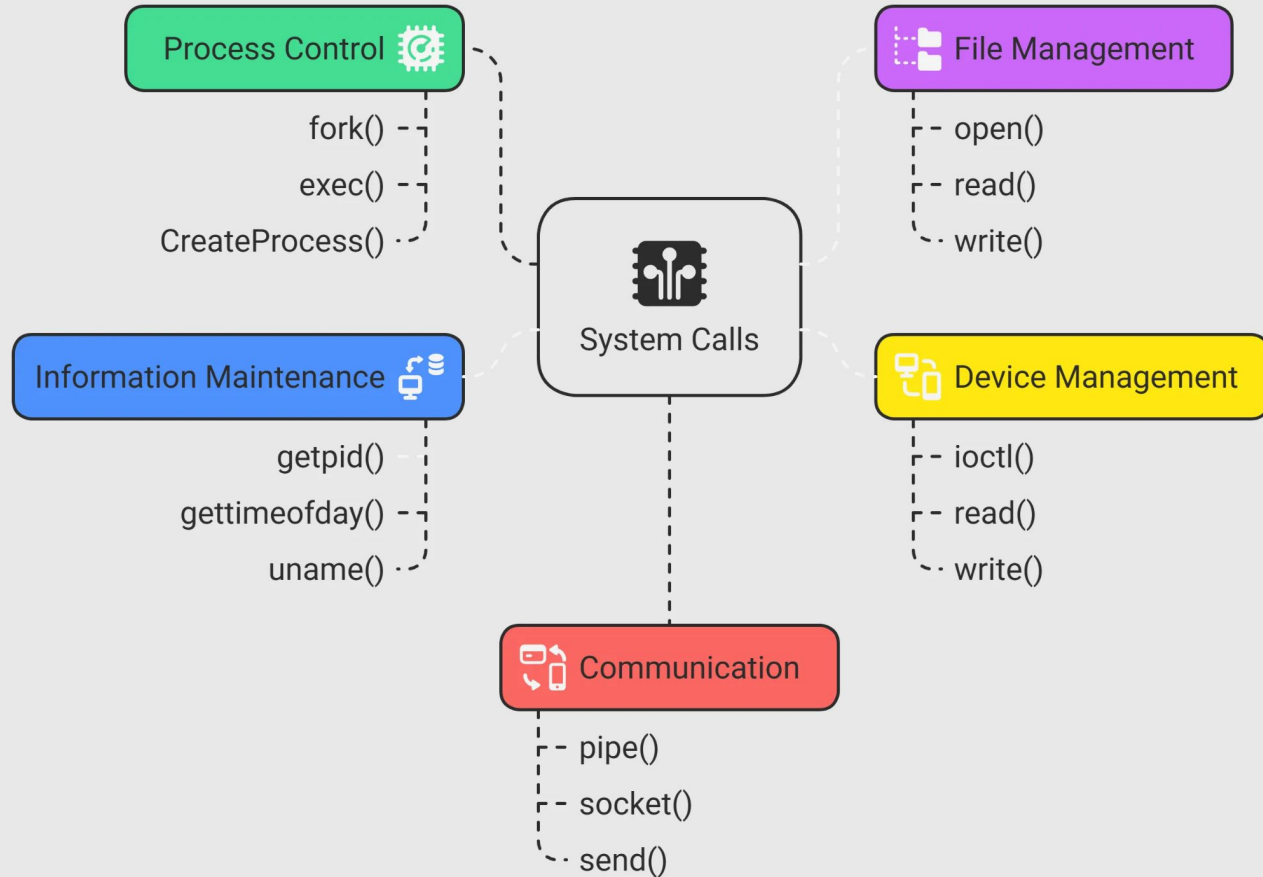


KERNEL MODE



System calls

System Call Types and Their Applications



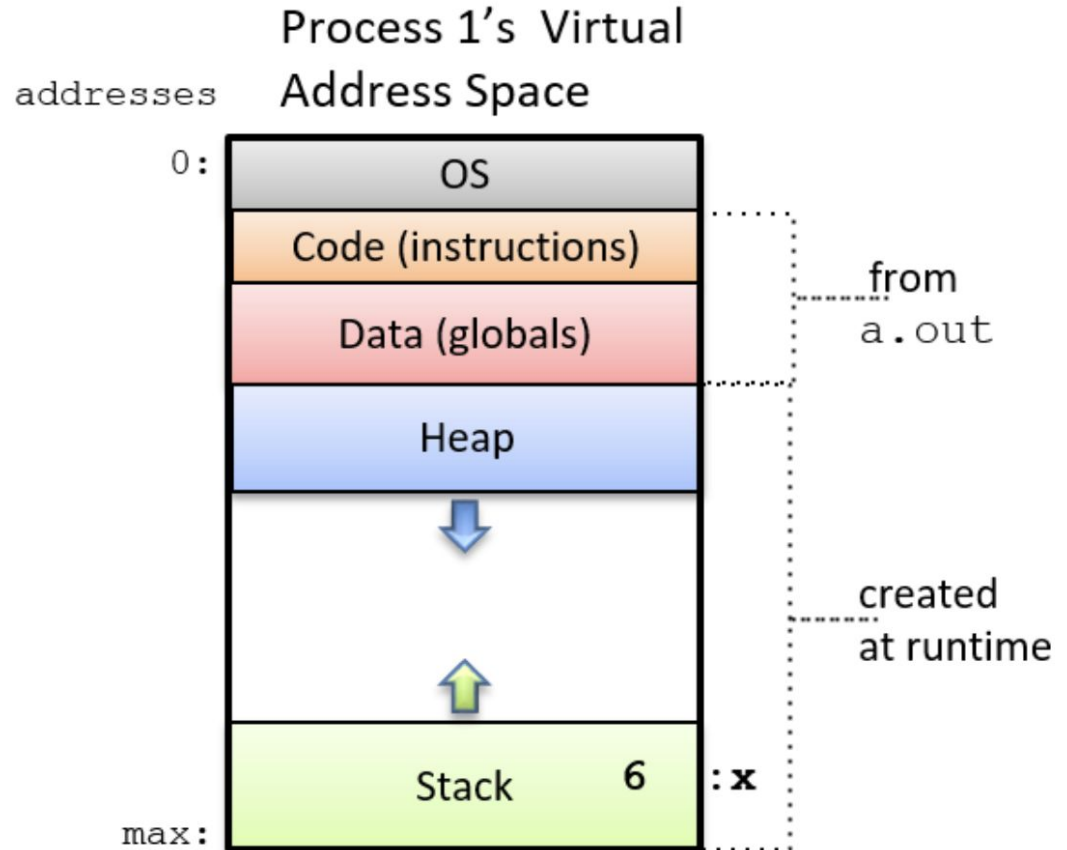
Process

- A process is a program that is currently being executed by the OS.
- When you run an exe file, the OS creates a process = a running program
- OS timeshares CPU across multiple processes: **virtualizes CPU**
- OS has a CPU scheduler that picks one of the many active processes to execute on a CPU

What Constitutes a Process?

- Every process has a unique process identifier (PID)
- Process occupies some memory in RAM (memory image)
 - *Code+data from executable*
 - *Stack+heap allocated for runtime use*
 - *Stack → function execution;*
 - *heap → dynamic allocation.*
- The execution context of the process (values of CPU registers)
 - *PC has address of instruction of process, some registers have process data*
 - *Process context is in CPU registers when process is running on CPU*
 - *Context saved in memory when process is paused, restored when run again*
- Ongoing communication with I/O devices
 - *Information is maintained about files that are open, ongoing network connections, other active connections to I/O devices*

Virtual address space



Execution context of the process (values of CPU registers)

1. **Program Counter (PC)**
 - a. Stores the address of the **next instruction** the process should execute. This tells the CPU where to continue.
2. **General-purpose registers**
 - a. Hold temporary process data, intermediate calculation results, function arguments, pointers, etc.
3. **Stack Pointer (SP)**
 - a. Points to the current position of the process's **stack**, which is needed to resume function calls correctly.
4. **CPU Flags / Status Register**
 - a. Stores CPU state such as zero, carry, sign, overflow, etc. Required to continue conditional instructions correctly.
5. **Process Control Block (PCB)**
 - a. OS data structure containing the process's saved CPU context along with other process information such as state, PID, scheduling information, etc.

How is a Program Loaded?

1. Program exists on disk
 - a. *The executable file contains machine code, data, metadata, and an entry point.*
2. User executes the program
3. OS creates a process
 - a. *The OS creates process-related information such as:*
 - i. PID
 - ii. **Process state**
 - iii. CPU registers
 - iv. File descriptors
 - v. Scheduling information
4. Virtual address space is created
5. Executable is loaded/mapped
 - a. *The OS loader maps the program's code and data into the process's virtual address space.
Required shared libraries are also mapped.*
6. Process becomes READY
7. CPU executes it

Summary & Important Questions

Summary

1. What is OS? Functions of OS?
2. Virtualization vs Abstraction
 - a. *VM vs Containerization*
3. System Call
 - a. *User mode vs Kernel mode*
4. What is Process? What Constitutes a Process?
5. Execution context of the process?
6. How is a Program Loaded?

Summary

1. What is OS? Functions of OS?

- a. *An Operating System (OS) is system software that manages hardware/resources and provides services and an interface for applications; its key functions include process, memory, file, I/O, storage, security, and resource management.*

2. Virtualization vs Abstraction

- a. *Abstraction hides complexity, while virtualization creates a virtual version of a physical resource so multiple isolated environments can share it.*
- b. *VM vs Containerization*
 - i. *A VM virtualizes the entire machine including the OS/kernel, whereas a container virtualizes the application environment while sharing the host OS kernel.*

3. System Call

- a. *A system call is the controlled interface through which a user-space program requests services from the OS kernel, such as `read()`, `write()`, or `fork()`.*
- b. *User mode vs Kernel mode*
 - i. *User mode restricts applications from directly accessing privileged resources, while kernel mode gives the OS full access to hardware and system resources.*

- What is Process? What Constitutes a Process?
 - *A process is a program in execution, consisting of its code, data, stack, heap, CPU execution context, and OS-managed metadata such as the PCB.*
- Execution context of the process?
 - *The execution context is the CPU state required to resume a process, mainly including the Program Counter, CPU registers, stack pointer, and processor status.*
- How is a Program Loaded?
 - *The OS creates a process, creates its virtual address space, maps/loads the program's code and data, initializes the stack and heap, sets the Program Counter to the entry point, and starts execution.*

States of Process

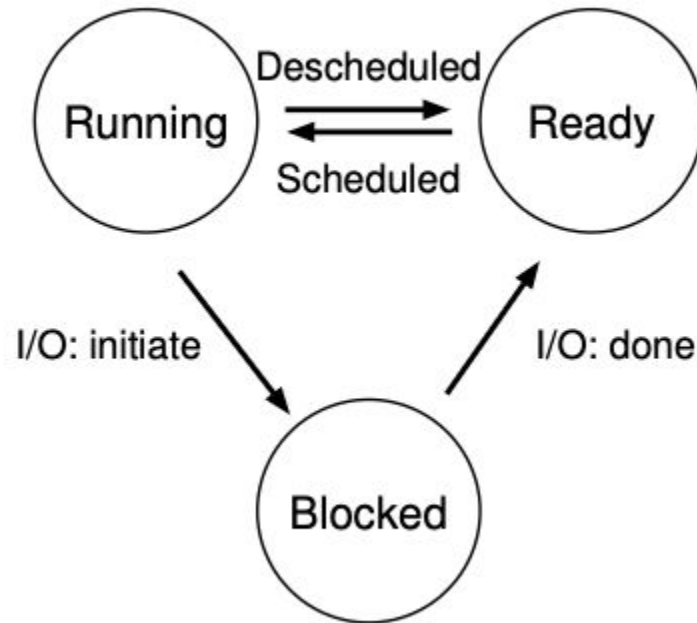
3 States of a process

OS manages multiple active processes at the same time. An active process can be in one of the following situations.

1. **Running:** currently executing on CPU
 - a. *CPU registers contain context of process*
2. **Blocked/suspended/sleeping:** process cannot run for some time
 - a. *Example: process has requested data from disk, command issued, but process cannot proceed until the data from disk is available*
3. **Ready/runnable:** ready to run but waiting for OS scheduler to switch the process in
 - a. *Many processes can be ready but scheduler can only run one on a CPU core*

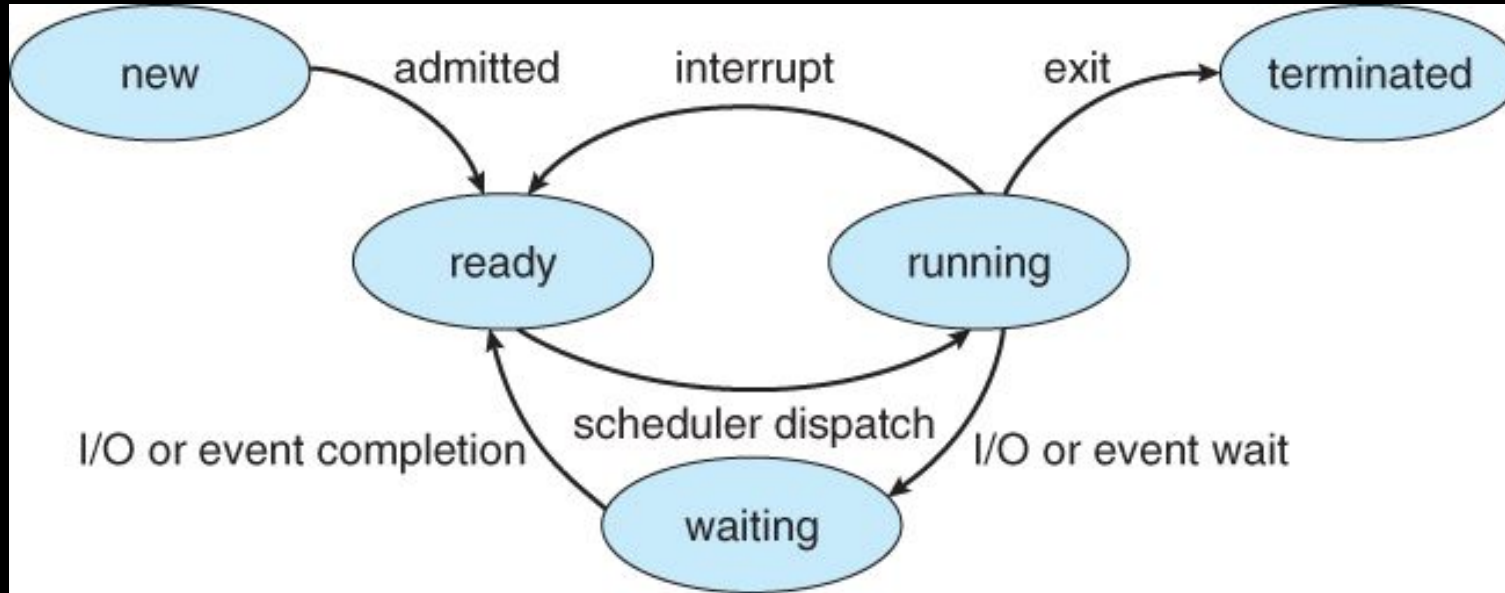
Context of blocked and ready processes is saved in OS memory, so that they can continue to run later on

3 state process model



5-state process model

- Important: Refer this model for exams, GATE Preparation



5-state process model

State	Meaning
New	Process is being created
Ready	Ready to execute, waiting for CPU
Running	Currently executing
Waiting/Blocked	Waiting for I/O/event
Terminated	Finished execution

GATE CSE 2024 — Process States

Question: Which of the following process state transitions is/are NOT possible?

- A. Running $\rightarrow$ Ready
- B. Waiting $\rightarrow$ Running
- C. Ready $\rightarrow$ Waiting
- D. Running $\rightarrow$ Terminated

GATE CSE 2024 — Process States

READY

"I am ready, but waiting for CPU."

RUNNING

"I currently have CPU."

WAITING

"I cannot proceed until some event/I/O happens."

TERMINATED

"I am finished."

Answer: B, C

GATE CSE 2020 — Process States

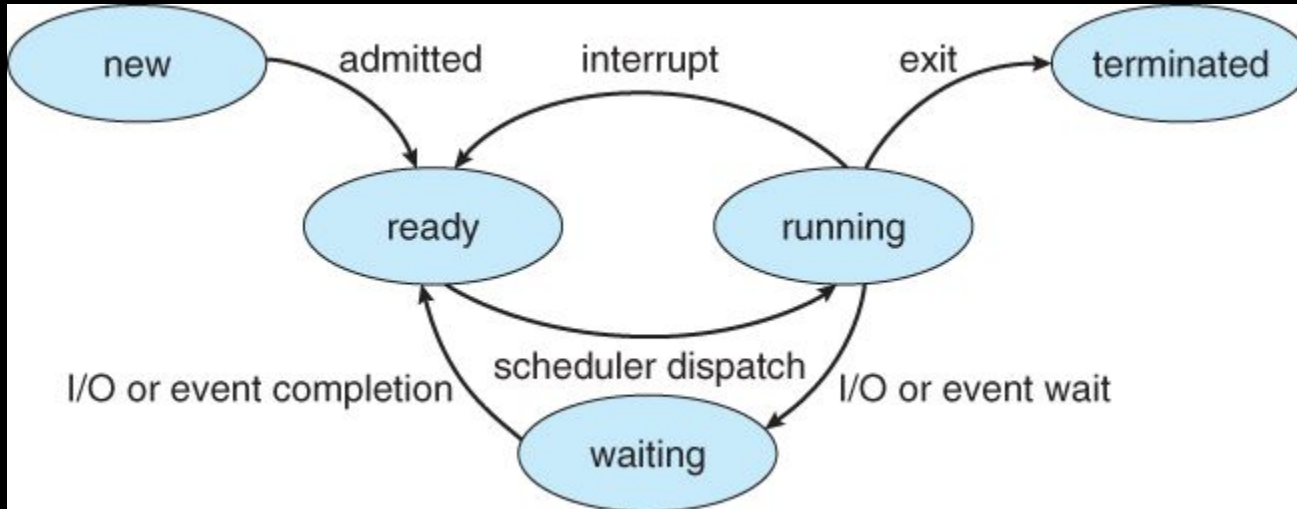
Consider the following statements about process state transitions for a system using preemptive scheduling.

- I. A running process can move to ready state.
- II. A ready process can move to running state.
- III. A blocked process can move to running state.
- IV. A blocked process can move to ready state.

GATE CSE 2020 — Process States

Answer: I, II and IV only

Always: Blocked (Waiting) → Ready → Running

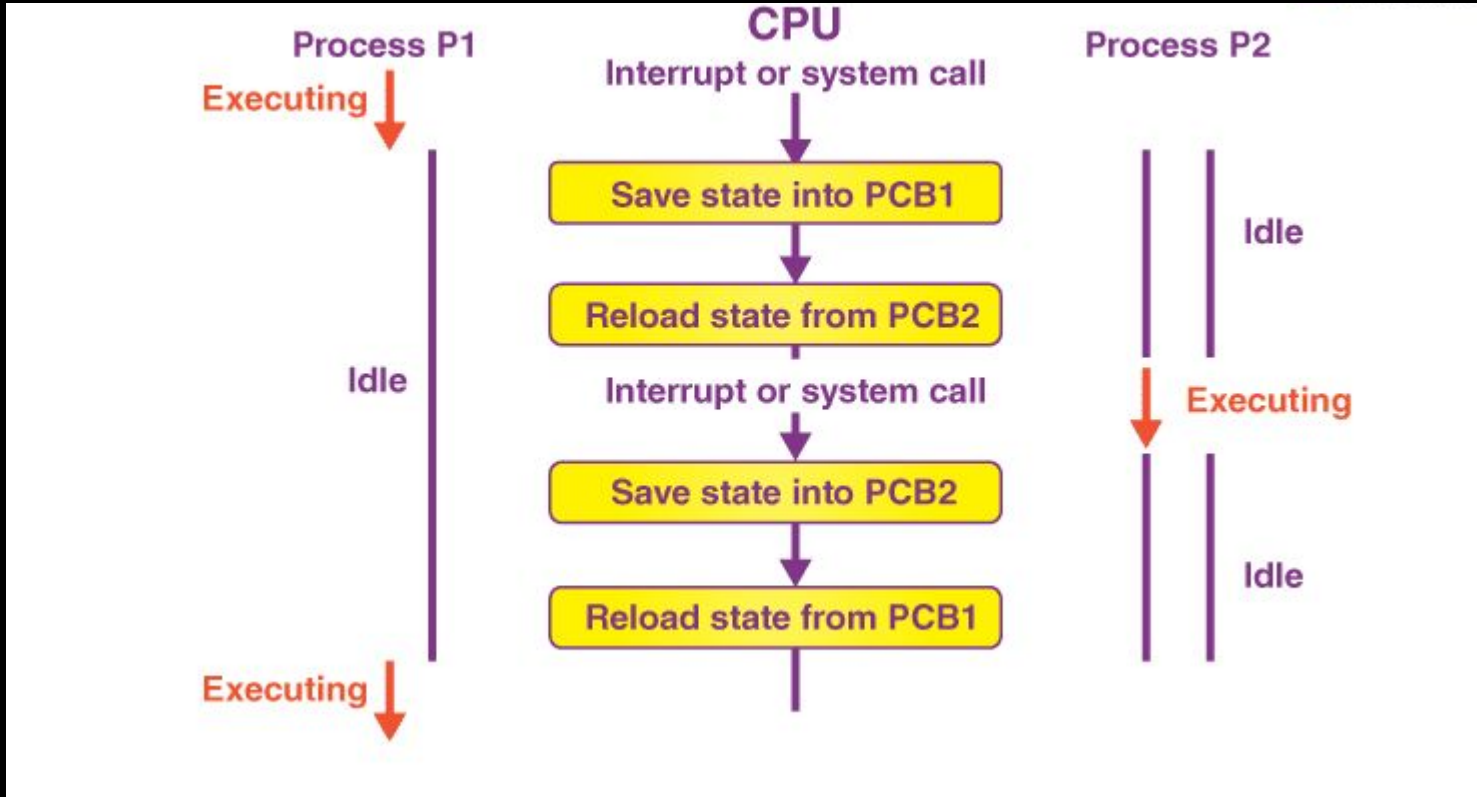


Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process ₀ initiates I/O
4	Blocked	Running	Process ₀ is blocked,
5	Blocked	Running	so Process ₁ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process ₁ now done
9	Running	–	
10	Running	–	Process ₀ now done

Figure 4.4: Tracing Process State: CPU and I/O

Context Switch

Context switch = switching the CPU from one process/thread to another.



What happens during a context switch?

- The entire process is NOT copied during a context switch.
- Code, heap, stack, and data generally remain in memory.
- The OS mainly switches the CPU execution context.
- Switching between processes may also require changing the memory/address-space context.

Context Switch vs Mode Switch

Context Switch

Process/Thread A → B

Changes execution context

Performed by OS scheduler

Saves/restores CPU state

Has scheduling overhead

Mode Switch

User Mode → Kernel Mode

Changes CPU privilege level

Caused by system call, interrupt, exception

Same process can continue

Does not necessarily switch process

GATE 2011 — Mode Switch vs Context Switch

Let T_1 be the time taken to switch from user mode to kernel mode, and T_2 be the time taken to switch from one user process to another user process. What is the relationship?

A. $T_1 > T_2$

B. $T_1 = T_2$

C. $T_1 < T_2$

D. Cannot be determined

GATE 2011 — Mode Switch vs Context Switch

Answer: C. $T_1 < T_2$

Why is $T_1 < T_2$?

- Because a mode switch requires less state to change.
- For mode switch, the CPU changes privilege level and enters the kernel through the appropriate mechanism. The same process remains active.
- For context switch, the OS has to switch the execution context. This is more work

Process Control Block

All information about a process is stored in a kernel data structure called the process control block (PCB)

- • Process identifier (PID)
- • Process state (running, ready, blocked, terminated, ..)
- • Pointers to other related processes (parent, children)
- • Saved CPU context of process when it is not running
- • Information related to memory locations of a process
- • Information related to ongoing I/O communication

PCB is known by different names in different OS

- • struct proc in xv6
- • task_struct in Linux

CPU Scheduler

Every OS maintains list of PCBs in some data structure

- Array, linked list, heap – what is suitable when?
- Scheduler loops through list of PCBs and picks process to run, switches to process, switches to another process after some time, and this continues...
- Scheduler picks one process to run on every core, so number of running process is the number of parallel processors available

GATE CSE 2023

Question: When switching from thread T1 to thread T2 of the same process, which need to be saved?

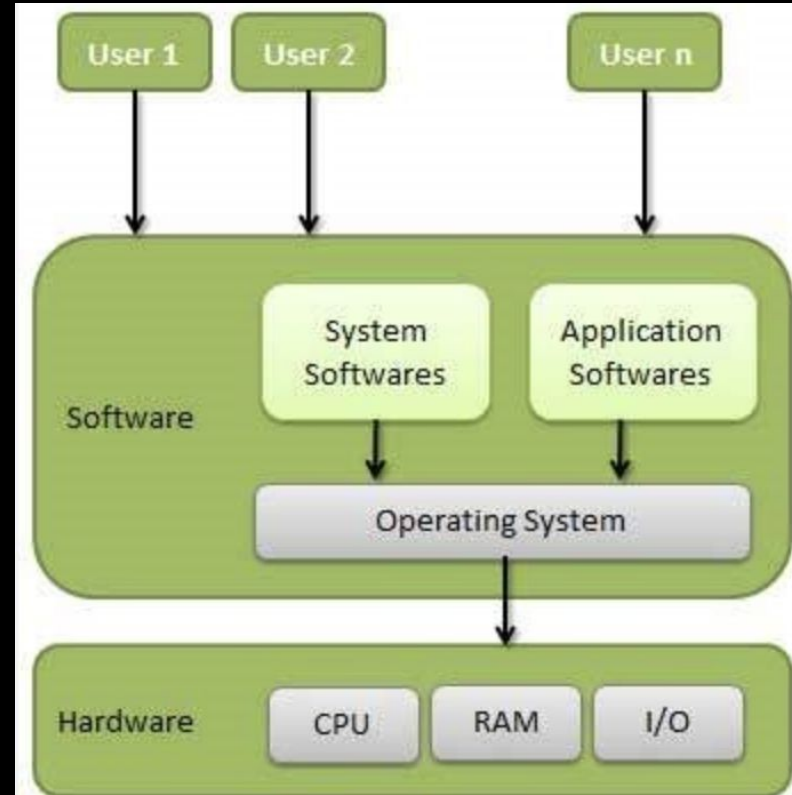
1. Page table base register
2. Stack pointer
3. Program counter
4. General-purpose registers

GATE CSE 2023

Answer: 2, 3, 4

Active Recall

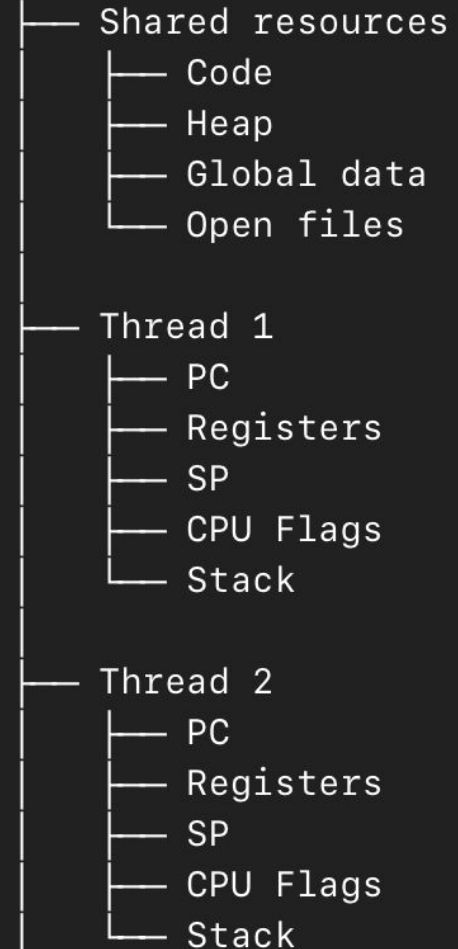
- Disk, Memory, CPU
- What constitutes a Process?
- Execution Context of Process
- Process lifecycle
- VM vs Container
 - *VM = Virtualizes entire Machine, including kernel*
 - *Container = Virtualize application environment, while sharing kernel*



Process vs Thread

1. Processes are usually **independent**, while threads exist as **subsets of a process**.
2. Each process has its own memory space. Threads that belong to the same process **share the same memory**.
3. A process is a **heavyweight operation**. It takes more time to create and terminate.
4. Context switching is more expensive between processes.
5. Inter-thread communication is faster for threads.

Process

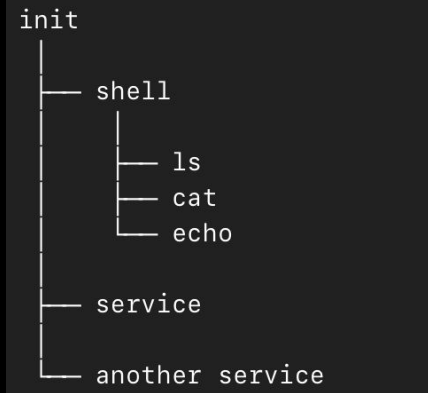


Process APIs

How does a shell work?

Working of Shell

- After bootup, the **init process is first process created**
- The init process spawns a shell like bash
- **All future processes are created by forking from existing processes like init or shell**
- Shell reads user command, forks a child, execs the command executable, waits for it to finish, and reads next command
- Common commands like ls, echo, cat are all readily available executables that are simply exec-ed by the shell



What is Shell?

- A shell is a program that reads commands from the user and asks the OS to execute them.

What is init?

- init is a process. It is the first **user-space process**.

Note: The kernel gets control first, initializes the machine, and eventually starts the first user-space process.

Process APIs: API for Process Management

- What API does OS provide to user programs to manage processes?
 - *How to create, run, terminate processes?*
- API = Application Programming Interface
 - *functions available to write user programs*
- API provided by OS is a set of “system calls”
 - *System call is a function call into OS code that runs at higher CPU privilege level*
 - *Sensitive operations (e.g., access to hardware) are allowed only at a higher privilege level*
- Some “blocking” system calls cause the process to be blocked and context switched out (e.g., `read()` from disk), while others (e.g., `getpid()` to get PID) can return immediately

Process APIs: Should we rewrite programs for each OS?

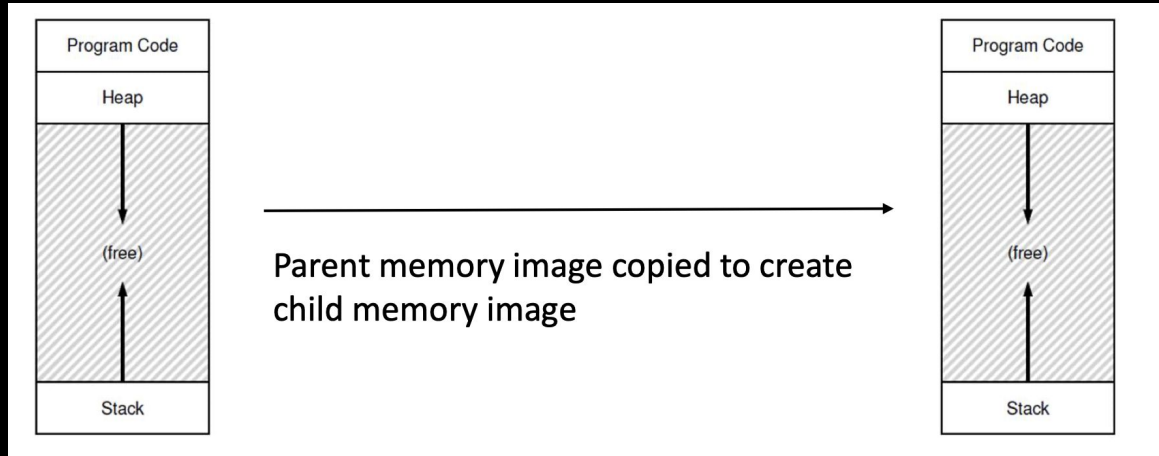
- POSIX API: a standard set of system calls that an OS must implement
 - *Programs written to the POSIX API can run on any POSIX compliant OS*
 - *Most modern OSes are POSIX compliant*
 - *Ensures program portability*
- Program language libraries hide the details of invoking system calls
 - *The printf function in the C library calls the write system call to write to screen*
 - *User programs usually do not need to worry about invoking system calls*

Process APIs: Process related system calls

- `fork()` creates a new child process
 - *All processes are created by forking from a parent*
 - *OS starts init process after boot up, which forks other processes*
 - *The init process is ancestor of all processes, including shell/terminal*
- `exec()` makes a process execute a given executable
- `exit()` terminates a process
- `wait()` causes a parent to block until child terminates
- Many variants of the above system calls exist in language libraries with different arguments

Process API: fork

- Parent process calls “fork” system call to create (spawn) a new process
- New child process created with new PID
- Memory image of parent is copied into that of child
- Parent and child run different copies of same code



Process API: What happens after fork?

- Parent and child resume execution in their copies of the code
- Child starts executing with a **return value of 0** from fork
- Parent resumes executing with a return value equal to child PID
- Parent and child run independently
- Any changes in parent's data after fork does not impact child

```
int ret = fork()
if(ret == 0) {
    print "I am child"
}
else if(ret > 0) {
    print "I am parent"
}
```

Child resumes
here

Parent resumes
here

```
int ret = fork()
if(ret == 0) {
    print "I am child"
}
else if(ret > 0) {
    print "I am parent"
}
```

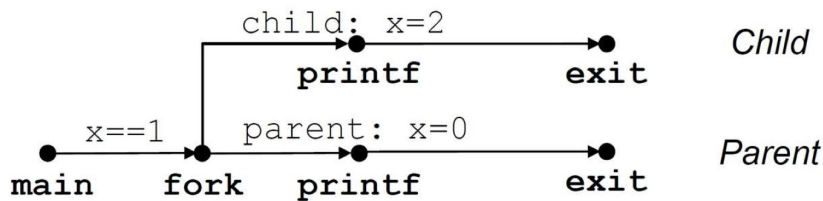
Process API: fork

- What values of x are printed?

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5      int ret = fork();
6      int x = 1;
7
8      if (ret == 0) {
9          printf("I am child\n");
10
11          x = x + 1;
12
13          printf("Child x = %d\n", x);
14      }
15      else if (ret > 0) {
16          printf("I am parent\n");
17
18          x = x - 1;
19
20          printf("Parent x = %d\n", x);
21      }
22      else {
23          printf("fork failed\n");
24      }
25
26      return 0;
27 }
```

Process API: fork

- What values of x are printed?
- Parent and child both start with their own independent copies of variable x in their memory images
- Child increments its copy of x, prints 2
- Parent decrements its copy of x, prints 0



```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     int ret = fork();
6     int x = 1;
7
8     if (ret == 0) {
9         printf("I am child\n");
10
11         x = x + 1;
12
13         printf("Child x = %d\n", x);
14     }
15     else if (ret > 0) {
16         printf("I am parent\n");
17
18         x = x - 1;
19
20         printf("Parent x = %d\n", x);
21     }
22     else {
23         printf("fork failed\n");
24     }
25
26     return 0;
27 }
```

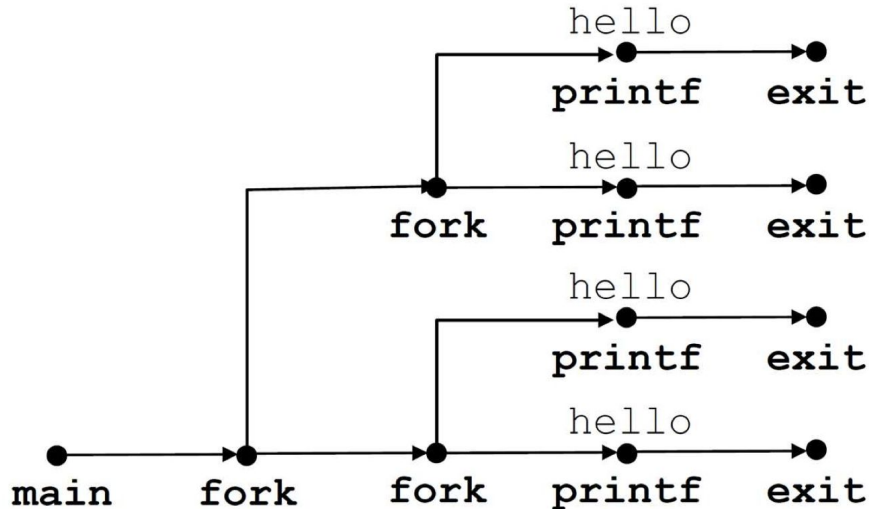
Process API: fork

- Total Processes?
- Number of Child processes?
- How many times hello gets printed?

```
fork()  
fork()  
print hello  
exit
```

Process API: fork

- Total Processes?
- Number of Child processes?
- Total 4 processes (1 parent + 3 child)
- Hello printed 4 times



```
fork()
fork()
printf hello
exit
```

Process API: GATE PYQ 2012

Consider the following code:

The total number of child processes created is:

1. 3
2. 4
3. 7
4. 8

```
fork();  
fork();  
fork();
```

Process API: GATE PYQ 2012

Consider the following code:

The total number of child processes created is:

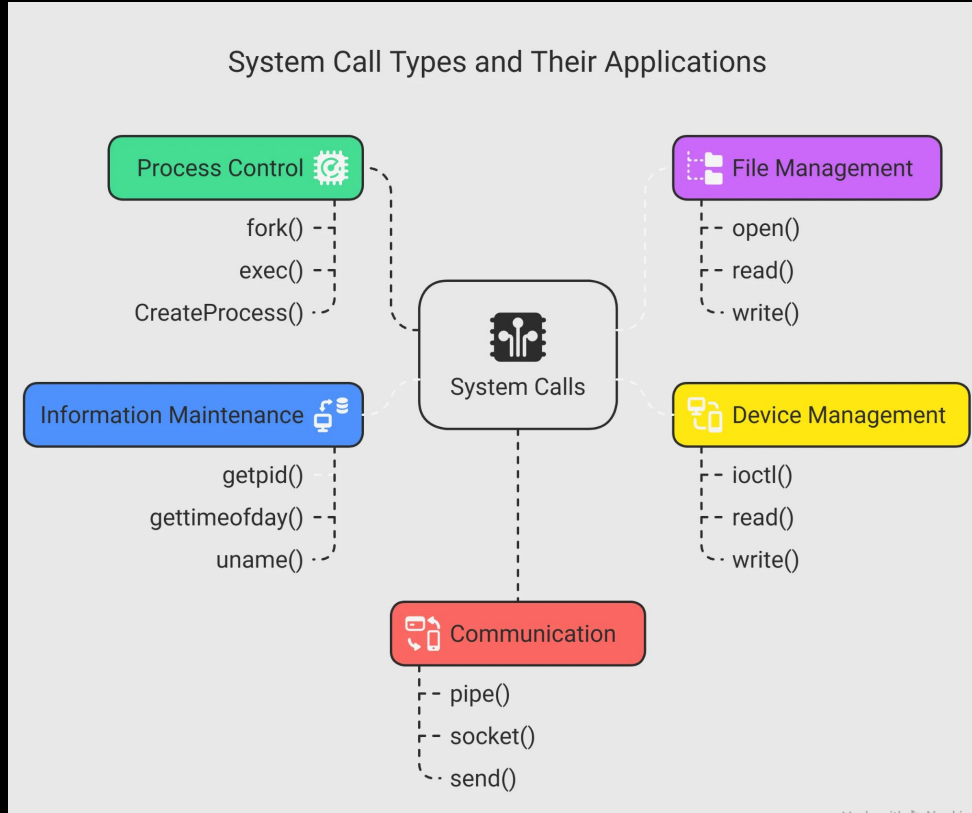
- A. 3
- B. 4
- C. 7
- D. 8

```
fork();  
fork();  
fork();
```

Answer: C. 7

8 total - 1 original parent = 7 children

System Calls other than Process



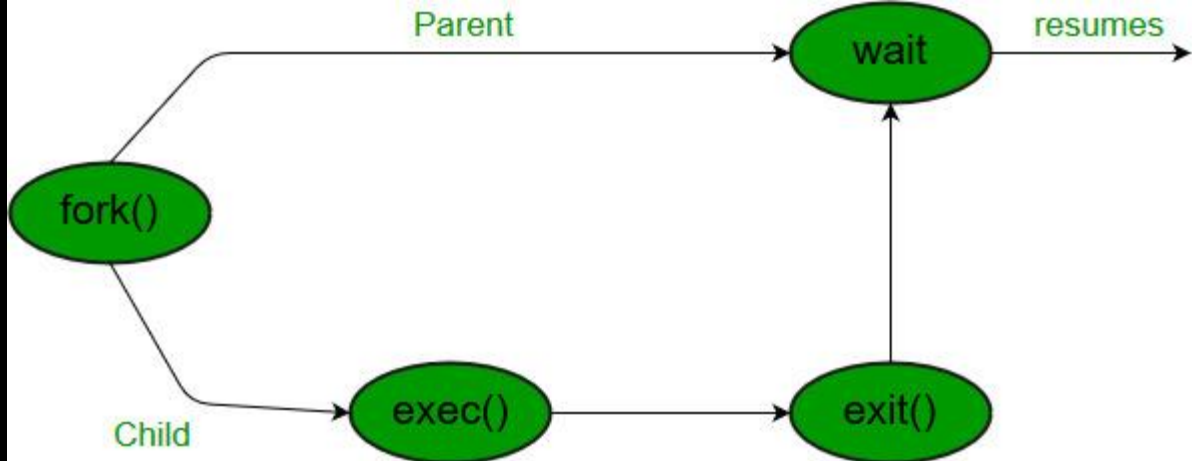
Process API: exec system call

What happens during exec?

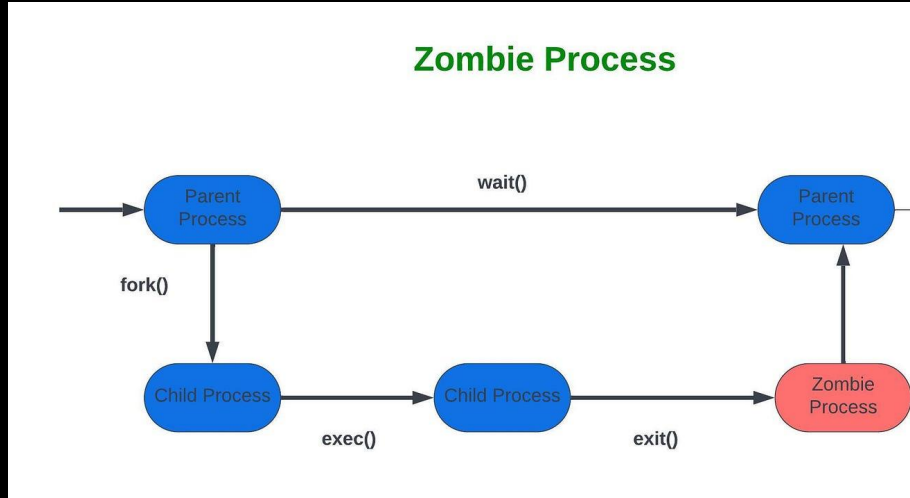
- After fork, parent and child are running same code
 - *Not too useful!*
- A process can run `exec()` to load another executable to its memory image
 - *So, a child can run a different program from parent*
- Variants of `exec()`, e.g., to pass command line arguments to new executable

Process API: wait system call

- Parent calls wait system call to reap (clean up memory of) a **zombie child**
- Wait cleans up memory of one terminated child and returns in parent process
 - If child still running, **wait system call blocks parent until child exits**
 - If child terminated already, **wait reaps child** and returns immediately
- If parent with no child calls wait, it returns immediately without reaping anything



Process: Zombie Process

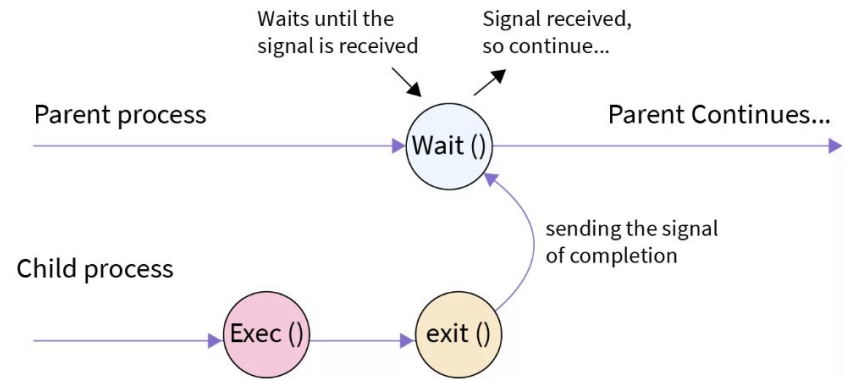


Process table		
Parent PID	→	Parent PCB
...	...	...
Child ID	→	Child PCB
...	...	...

- A zombie process is a dead child process that has finished running, but its entry stays in the system **process table** because the parent process has not yet read its exit status.
- Note: **Process table is a data structure in RAM** to store information about a process.

Process: wait system call

- Wait system call & Zombie process



Before exit() system call

Process table		
Parent PID	→	Parent PCB
...	...	...
Child ID	→	Child PCB
...	...	...

After exit() system call

Process table		
Parent PID	→	Parent PCB
...	...	...
Child pID	→	Child PCB
...	...	...

Zombie process



its indicates that the child process is done with its execution & entered into 'Zombie State'

After exit() system call

Process table		
Parent PID	→	Parent PCB
...	...	...
Child PID	→	Child PCB
...	...	...

Zombie Gone



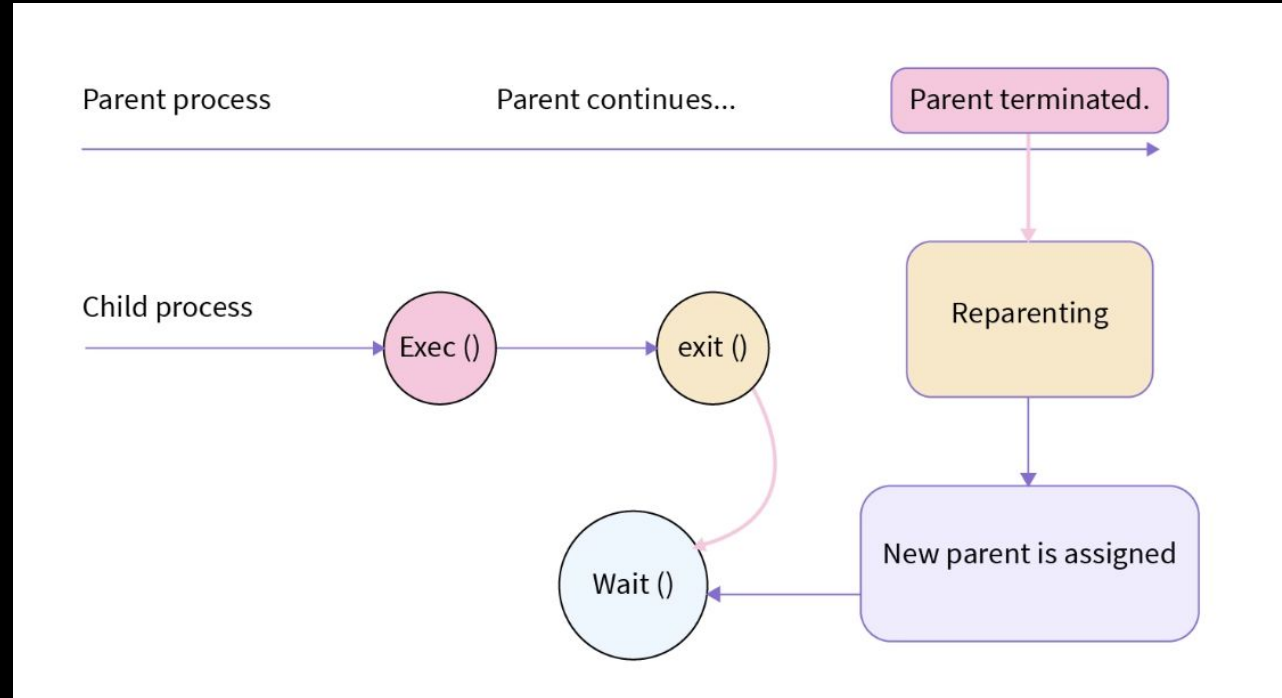
Once the wait() system call is called by the parent, it reads the exit status of the child process and reaps it from the process table

Process API: wait system call

- Wait system call variant `waitpid` reaps a specific child with a given PID, while regular `wait` reaps any terminated child
- Wait system call “reaps” one dead child at a time (in any order)
 - *Every fork must be followed by call to wait at some point in parent*
- What if parent has exited while child is still running?
 - *Child will continue to run, becomes orphan*
 - *Orphans adopted by init process, reaped by init when they terminate*
- If parent forks children, but does not bother calling `wait` for long time, system memory fills up with zombies
 - *Common programming error, exhausts system memory*

Process APIs: Orphan process

- Reparenting
- Parent is dead



Process API: exit system call

- When a process finishes execution, it calls exit system call to terminate
- OS switches the process out and never runs it again
- Exit is automatically called at end of main
- Exiting process cannot clean up its memory, and memory must be freed up by someone else
- Terminated process exists in a zombie state

Process API: Summary

1. fork
2. Exec
3. Wait
4. Zombie process
5. Orphan process
6. init

Function call vs System call

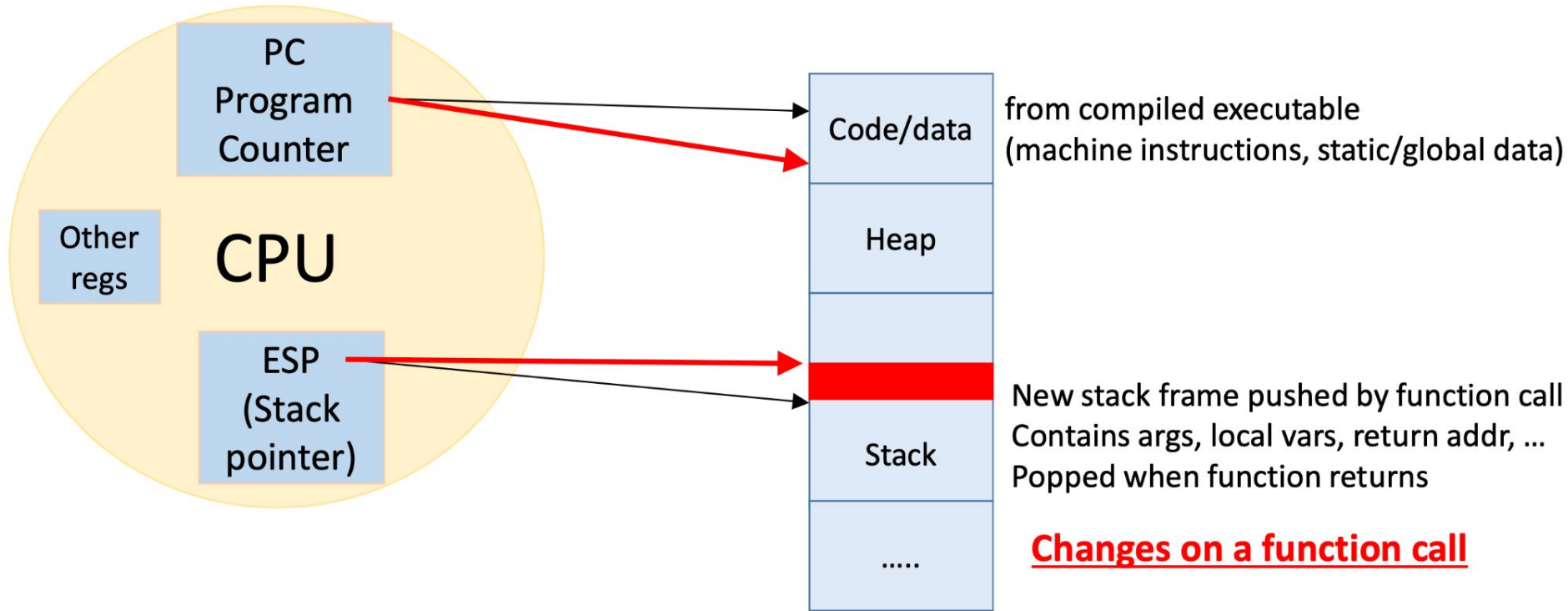
What happens when a user program makes a function call?

- Allocate memory on user stack for function arguments, local variables, ..
- Push return address, PC jumps to function code
- Push register context (to resume execution when function returns)
- Execute function code
- When returning from function, pop return address, pop register context

System call also must

- Use a stack to push register context
- Save old PC, change PC to point to OS code to handle system call
- Run system call, restore context back to user code

Function Call



Located at some memory addresses in RAM

System Call

What is different for a system call?

Changing PC in function call vs. system call

- In function call, address of function code known in executable, can jump to function code directly using a CPU instruction (“call” in x86)
- For system call, cannot trust user to jump to correct OS code (what if user jumps to inappropriate privileged code?)

Saving register context on stack in function call vs. system call

- In function call, register context is saved and restored from user stack
- For system call, OS does not wish to use user stack (what if user has setup malicious values on the stack?)

We require: a **secure stack**, a secure way of jumping to OS code

Kernel Stack and IDT

Every process uses a separate **kernel stack** for running kernel code

- Part of PCB of process, in OS memory, not accessible in user mode
- Used like user stack, but for kernel mode execution
- Context pushed on kernel stack during system call, popped when done

To set PC, CPU accesses **Interrupt Descriptor Table (IDT)**

- Data structure with addresses of kernel code to jump to for events
- Set up by OS during boot up, not accessible in user mode
- CPU uses IDT to locate address of OS code to jump to

Together: secure way of locating OS code, secure stack for OS to run

Trap instruction

When user code wants to make system call, it invokes special “trap instruction” with an argument

- Example: “int n” in x86, argument “n” indicates type of trap (syscall, interrupt)
- The value of “n” specifies index into IDT array, which OS function to jump to

When CPU runs the trap instruction:

- CPU moves to higher privilege level
- CPU shifts stack pointer register to kernel stack of process
- Register context is saved on kernel stack (part of PCB)
- Address of OS code to jump to is obtained from IDT, PC points to OS code
- OS code starts to run, on a secure stack

Why do we need trap instruction?

Need a secure way of jumping to OS code to handle traps

- User code cannot be trusted to jump to correct OS code
- Only CPU can be trusted to handover control from user to OS securely

Who calls trap instruction?

- System call code in a language library (printf invokes system call via int n)
- External hardware raises interrupt, causes CPU to execute “int n”
- Argument “n” indicates whether system call /IRQ number of hardware device

Across all cases, the mechanism is: save context on kernel stack, switch to OS address in IDT, run OS code to handle trap

Return from trap

When OS is done handling syscall or interrupt, it calls a special instruction return-from-trap

- Restore context of CPU registers from kernel stack
- Change CPU privilege from kernel mode to user mode
- Restore PC and jump to user code after trap

User process unaware that it was suspended, resumes execution at the point it stopped before

Always return to the same user process from kernel mode? No

- Before returning to user mode, OS checks if it must switch to another process

Function call vs System call

- Function call: application calls another function.
- System call: application requests a service from the OS.

	Normal function call	System call
Called by	Program	Program
Executes in	User code	Kernel code
Privilege	User mode	Kernel mode
CPU privilege transition	No	Yes
Kernel involved	No	Yes
Example	<code>add()</code>	<code>fork()</code> , <code>read()</code> , <code>write()</code>
Why?	Reuse application code	Request OS service

CPU Scheduling Policies

Why switch between processes?

Sometimes when OS is in kernel mode, it cannot return back to the same process that was running in user mode before

- Process has exited or must be terminated (e.g., segfault)
- Process has made a blocking system call

Sometimes, the OS does not want to return back to the same process

- The process has run for too long
- Must timeshare CPU with other processes

In such cases, OS performs a context switch from one process to another

- Switch from kernel mode of one process to kernel mode of another
- OS scheduler decides which process to run next and switches to it

OS scheduler

- OS maintains list of all active processes (PCBs) in a data structure
 - *Processes added during fork, removed after clean up in wait*
- OS scheduler is special code in the OS that periodically loops over this list and picks processes to run
- Basic outline of scheduler code
 - *When invoked, save context of currently running process in its PCB*
 - *Loop over all ready/runnable processes and identify a process to run next*
 - *Restore context of new process from PCB and get it to run on CPU*
 - *Repeat this process as long as system is running*

OS Scheduler

- OS scheduler schedules process on CPU
 - *One process at a time per core*
 - *Multiple processes can run in parallel on multiple cores*
- Scheduling policy: which one of the ready/runnable processes should be run next on a given CPU core?
- Mechanism of context switching (save context of old process in its kernel stack/PCB and restore context of new process) is independent of policy
- Simple scheduling policies have good theoretical guarantees, but not practical for real operating systems
- Real-life schedulers are very complex, involve many heuristics

Scheduling and context switching

- OS scheduling involves two tasks
 - *Policy to pick which process to run*
 - *Mechanism to switch to that process*
- Non preemptive (cooperative) schedulers are polite
 - *Switch only if process blocked or terminated*
- Preemptive (non-cooperative) schedulers can switch even when process is ready to continue
 - *CPU generates periodic timer interrupt*
 - *After servicing interrupt, OS checks if the current process has run for too long*

Preemptive vs. non preemptive schedulers

When is the OS scheduler invoked to trigger a context switch?

- Only when a process is in kernel mode for a trap, but not on every trap

Non-preemptive scheduler performs only voluntary context switches

- Process makes blocking system call
- Process has exited or has been terminated

Preemptive scheduler performs involuntary context switches also

- Process can be context switched out even if process is still runnable/ready
- OS can ensure that no process runs for too long on CPU, starving others
- Modern systems use preemptive schedulers
- Process can be context switched out any time in its execution

Timer interrupts

- Scheduler (OS) can run only when process is in kernel mode
 - *What if process never goes to kernel mode? How to perform pre-emptive scheduling?*
- Timer interrupts: special interrupts that go off periodically
- Used by OS to get back in control periodically
 - *Increment clock ticks, do other book keeping*
 - *Run scheduling algorithm if required to do involuntary context switch of currently running process*
- Hardware support in the form of timer interrupts essential to implementing pre-emptive scheduling policies

Goals of CPU scheduling policy

- Maximize utilization: efficient use of CPU hardware
- Minimize completion time / turnaround time of a process = time from process creation to completion
- Minimize response time of a process = time from process creation to first time it is run (important for interactive processes)
- Fairness: all processes get a fair share of CPU (account for priorities also)
- Low overhead of scheduling policy
 - *Scheduler does not take too long to make a decision (even with large #processes)*
 - *Scheduler does not cause too many context switches (~1 microsecond to switch)*

Simplest policy: First In First Out

- Newly created processes are put in a FIFO queue, scheduler runs them one after another from queue
- Non-preemptive: process allowed to run till it terminates or blocks
 - *When process unblocks, the next run is separate “job”, added to queue again*
 - *That is, if a process comes back after I/O wait, it counts as a fresh CPU burst (CPU burst = the CPU time used by a process in a continuous stretch)*
- Example schedule: P1 (1-5), P2 (6-8), P3 (9 to 10)

Process	CPU time needed (units)	Arrives at end of time unit	Execution time slots
P1	5	0	1-5
P2	3	1	6-8
P3	2	3	9-10

Problem with FIFO

- Example: three processes arrive at $t=0$ in the order A,B,C
- Problem: convoy effect (small processes get stuck behind long processes)
- Average turnaround times tend to be high

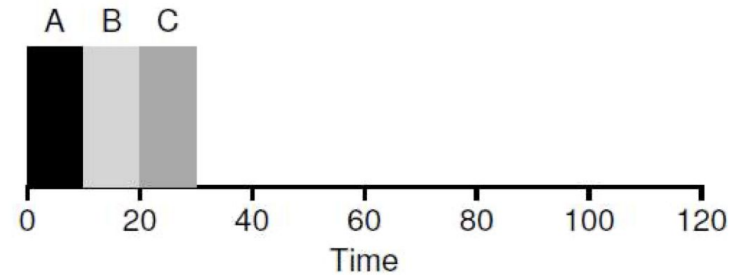


Figure 7.1: FIFO Simple Example

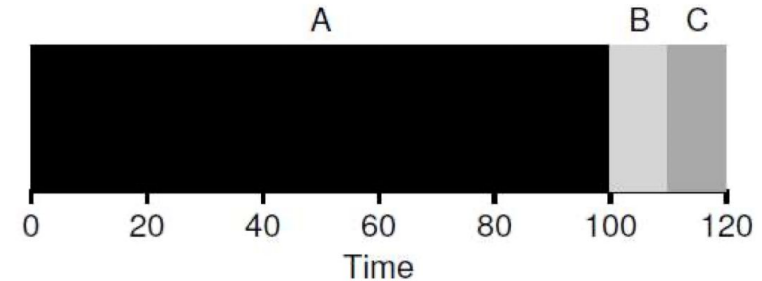


Figure 7.2: Why FIFO Is Not That Great

Shortest Job First (SJF)

- Assume CPU burst of a process (amount of time a process runs on CPU until termination/blocking) is known apriori
- Pick process with smallest CPU burst to run next, non-preemptive
 - *Store PCBs in a heap-like data structure, extract process with min CPU burst*
- Example schedule: P1 (1-5), P3 (6-7), P2 (8-10)

Process	CPU burst	Arrival time	Execution time slots
P1	5	0	1-5
P2	3	1	8-10
P3	2	3	6-7

Problem with SJF

- Provably optimal when all processes arrive together
- Theoretically guaranteed to have the lowest average turnaround time across all policies (under certain assumptions)
- SJF is non-preemptive, so short jobs can still get stuck behind long ones.

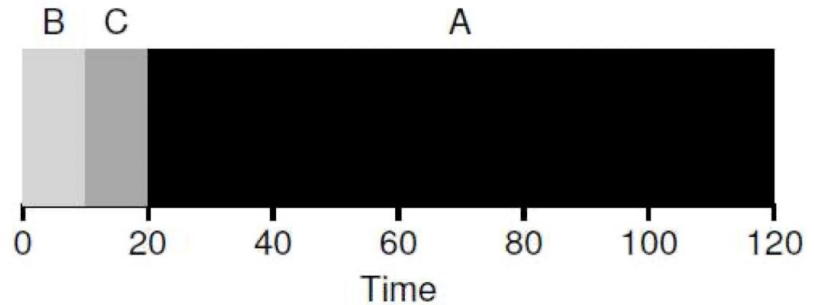


Figure 7.3: SJF Simple Example

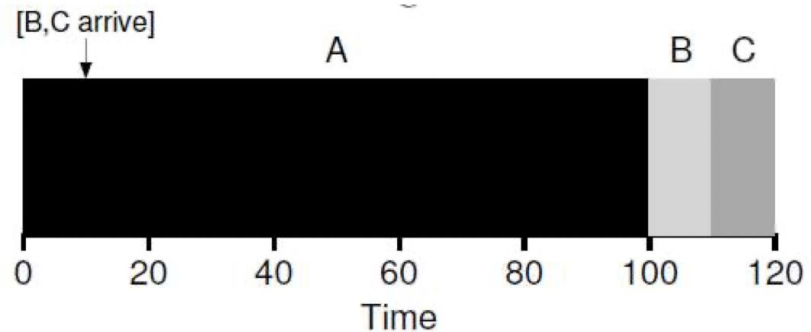


Figure 7.4: SJF With Late Arrivals From B and C

Shortest Remaining Time First (SRTF)

- Preemptive version of SJF A newly arrived process can preempt a running process, if CPU burst of new process is shorter than remaining time of running process
- Avoids problem of short process getting stuck behind long one
- Example schedule: P1 runs for 1 unit, P2 (2-4), P3 (5-6), P1 (7-10)

Process	CPU burst	Arrival time	Execution time slots
P1	5	0	1, preempted , then 7-10
P2	3	1	2-4
P3	2	3	5-6

Round Robin (RR)

- Every process executes for a fixed quantum slice
 - *Slice not too small (to amortize cost of context switch)*
 - *Slice not too big (to provide good responsiveness)*
- Preemptive policy
 - *Timer interrupt used to enforce periodic scheduling*
- Good for response time, fairness
- Bad for turnaround time
- xv6 is a simple RR scheduler

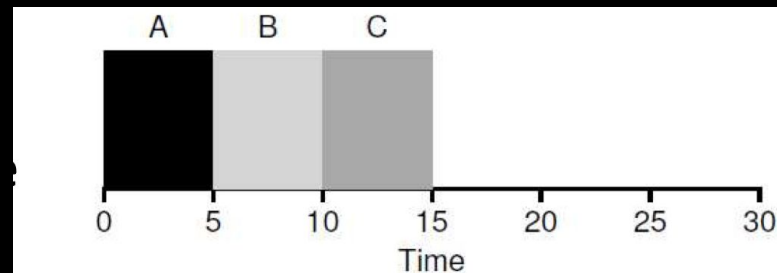


Figure 7.6: SJF Again (Bad for Response Time)

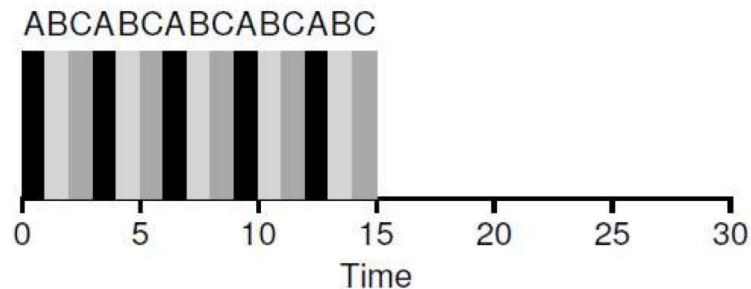


Figure 7.7: Round Robin (Good for Response Time)

Weighted Fair Queueing (WFQ)

- Round robin with different weights or priorities to processes
 - *Decided by scheduler or can be set by users*
 - *Time slice will be in proportion to the weight or priority*
- Real life schedulers may not be able to enforce time slice exactly
 - *What if timer interrupt is not exactly aligned with time slice?*
 - *What if process blocks before its time slice?*
- Practical modification: keep track of run time of process, schedule process that has used least fraction of its fair share
 - *Compensate excess/deficit running time in future time slices*
- Linux scheduler is a variant of weighted fair queueing
 - *CFS (completely fair scheduler) uses red-black trees to keep track of the fair share run time of processes, schedules the one with least run time*

Multi-level feedback queue (MLFQ)

- Linux uses a Multi Level Feedback Queue (MLFQ)
- Real schedulers are more complex
 - *Many queues, in order of priority*
 - *Process from highest priority queue scheduled first*
 - *Within same priority, any algorithm like RR*
 - *Priority of process decays with its age*

MLFQ Overview

- Multiple queues, one for each priority level
 - Schedule processes from highest priority queue to lowest
 - Use round robin scheduling for processes within same priority level

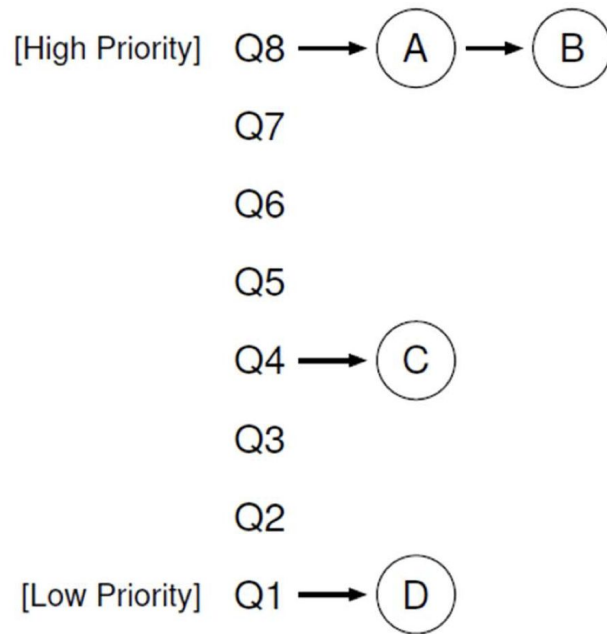


Figure 8.1: MLFQ Example

MLFQ Summary

- **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).
- **Rule 2:** If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR.
- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue).
- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).
- **Rule 5:** After some time period S , move all the jobs in the system to the topmost queue.

MLFQ: I/O & CPU bound processes

- Prioritize short, I/O bound processes over long, CPU bound processes
 - *Why? System works best when short processes don't have to wait for too long behind longer processes (that's why SJF is optimal!)*
 - *How is this done? I/O bound processes will stay at highest priority level*
 - *We do this without knowing the run times apriori, so practical to work in real life scenarios*
 - *Real life schedulers cannot make assumptions on knowing run times etc, but still try to achieve good properties of theoretically optimal schedulers*

GATE CSE 2026 Set 2

GATE CSE 2026 Set 2 — 1 Mark

Which one of the following CPU scheduling algorithms cannot be preemptive?

- A. Shortest Remaining Time First (SRTF) Scheduling
- B. First Come First Serve (FCFS) Scheduling
- C. Round Robin Scheduling
- D. Priority Scheduling

GATE CSE 2026 Set 2

GATE CSE 2026 Set 2 — 1 Mark

Which one of the following CPU scheduling algorithms cannot be preemptive?

- A. Shortest Remaining Time First (SRTF) Scheduling
- B. First Come First Serve (FCFS) Scheduling
- C. Round Robin Scheduling
- D. Priority Scheduling

Answer: B — FCFS

GATE CSE 2002

GATE CSE 2002 — 1 Mark

Which of the following scheduling algorithms is non-preemptive?

- A. Round Robin
- B. First-In First-Out
- C. Multilevel Queue Scheduling
- D. Multilevel Queue Scheduling with Feedback

GATE CSE 2002

GATE CSE 2002 — 1 Mark

Which of the following scheduling algorithms is non-preemptive?

- A. Round Robin
- B. First-In First-Out
- C. Multilevel Queue Scheduling
- D. Multilevel Queue Scheduling with Feedback

Answer: B — First-In First-Out (FIFO/FCFS)

Summary

1. **Scheduling Policies** can be preemptive and non-preemptive
 - a. *Scheduling policy = decision engine = which one to run first*
2. **Mechanisms** are context switches, timer interrupts, preemption, dispatcher
 - a. *Mechanisms = machinery or operations to run that process*
3. **Real OS Scheduling policy** is far more complicated than standard scheduling policies
 - a. *Real OS use multiple scheduling policies based on nature of task*
4. Preemptive and Non-Preemptive Scheduling is **Classification of Scheduling Algorithms**
 - a. *Preemptive*
 - b. *Non-Preemptive*
5. Why do we need Scheduler?
6. Some Scheduling algorithms have both preemptive and non-preemptive variants.

Summary

Scheduling algorithm	Non-preemptive	Preemptive	
FCFS	Yes	—	—
SJF	Yes	—	—
SRTF	—	Yes	Yes
Priority	Yes	Yes	Yes
Round Robin	—	Yes	Yes
Multilevel Queue	Depends	Depends	
Multilevel Feedback Queue	Typically preemptive	Yes	

Practice Question 1

Q1: Consider the following sequence:

A user starts Application A. The executable is stored on disk. The OS creates a process for A. A is scheduled on the CPU and starts running. While running, A requests data from a file. The OS services the request, and another ready process B gets the CPU.

Explain the OS concepts involved in this sequence.

Practice Question 1

- Program → Process
- Program loading / virtual address space
- System call / user mode → kernel mode
- Process state change and scheduling

Expected Answer

The executable initially exists as a program on disk. When the user runs it, the OS creates a process, including a PID, process state, memory image and execution context. The OS creates its virtual address space and maps the program's code and data into it.

The process starts in a state where it can be scheduled and eventually becomes Running. When Application A requests file data, it uses a system call, causing execution to enter kernel mode so the OS can perform the requested service.

If A must wait for the file I/O, A moves from Running → Blocked/Waiting. The scheduler can then select B, causing B to move from Ready → Running. When A's I/O completes, it can move back to Ready.

These concepts are presented across the slides on program loading, system calls, processes and process states.

Practice Question 2

Q2: A company wants to run three applications on one physical server.

Approach A: Each application runs with its own Guest OS on top of a hypervisor.

Approach B: Each application runs in a separate container while sharing the host OS kernel.

Explain the main architectural difference between A and B. Which approach provides full machine/OS virtualization, and which approach primarily provides process/environment isolation?

Practice Question 2

Expected Answer

Approach A represents virtual machines. Each VM has a Guest OS and runs above a hypervisor. The machine and its OS are virtualized.

Approach B represents containers. Containers run isolated application environments while sharing the host OS kernel. The slides describe containers as isolated environments using namespaces, while VMs virtualize the machine/OS layer.

Therefore:

A → Virtualization / VM

B → Containerization / isolation

Practice Question 3

Q3: Suppose Process P1 is running and the OS decides to stop it temporarily so that another process can run.

Explain what information the OS needs to preserve so that P1 can continue correctly later. Also explain why the entire process does not need to be copied during a context switch.

Practice Question 3

Expected Answer

The OS needs to preserve P1's CPU execution context, including the Program Counter, general-purpose registers, Stack Pointer, and processor status/flags. This information allows the CPU to resume P1 from the correct instruction with the correct register and stack state.

The OS does not need to copy the entire process because the process's code, data, heap and stack generally remain in memory. The main operation is saving P1's CPU state and restoring the state of the next process.

Practice Question 4

Q4: A process is executing in user mode and performs a system call. Later, the scheduler switches the CPU from Process P1 to Process P2.

Explain the difference between these two switches:

User mode → Kernel mode

P1 → P2

Why is the second operation generally more expensive?

Practice Question 4

Expected Answer

A mode switch changes the CPU's privilege level from user mode to kernel mode so that the kernel can execute a privileged service. The same process is still executing; only its execution mode changes.

A context switch changes the process/thread executing on the CPU. The OS must save the execution context of P1 and restore the saved context of P2. This can involve the program counter, stack pointer, general-purpose registers and, for a process switch, address-space-related state.

Therefore, a context switch generally involves more state changes than a simple mode switch.

Practice: CPU Scheduling

1. Calculation of Waiting time
2. Calculation of Turnaround time
3. Preemptive and Non-preemptive policies
4. Execution time slots
5. FCFS
6. SJF
7. SRTF
8. RR

Practice

Which combination correctly ranks scheduling algorithms from most preemptive-friendly to strictly non-preemptive: SRTF, Round Robin, FCFS?

1. FCFS, Round Robin, SRTF
2. SRTF and Round Robin are preemptive; FCFS is strictly non-preemptive
3. All three are strictly non-preemptive
4. All three are strictly preemptive

Practice: FCFS

Two processes P1 and P2 arrive at time 0 with burst times 5 ms and 9 ms respectively. Using FCFS scheduling (P1 before P2), compute the waiting time and turnaround time for each, and the average waiting time.

Given:

Process	Arrival	Burst
P1	0	5 ms
P2	0	9 ms

Practice

Process	Arrival	Burst	Completion	Waiting Time	Turnaround Time
P1	0	5	5	0	5
P2	0	9	14	5	14

Average WT = 2.5 ms

Practice: SJF

Four processes P1, P2, P3, P4 arrive at time 0 with burst times 6, 2, 5, 3 ms. Using SJF (non-preemptive) scheduling: (a) Compute the waiting time and turnaround time for each process. (b) Compute the average waiting time and average turnaround time.

Given:

Process	Arrival	Burst
P1	0	6 ms
P2	0	2 ms
P3	0	5 ms
P4	0	3 ms

Practice: SJF

Process	Arrival	Burst	Start	Completion	WT	TAT
P2	0	2	0	2	0	2
P4	0	3	2	5	2	5
P3	0	5	5	10	5	10
P1	0	6	10	16	10	16

Average WT = 4.25 ms

Average TAT = 8.25 ms

Practice: SRTF

Three processes P1, P2, P3 arrive at times 0, 1, and 3 ms with burst times 9, 5, and 2 ms respectively. Using preemptive SRTF scheduling: (a) Draw the Gantt chart. (b) Compute waiting and turnaround time for each process. (c) Compute the average waiting time.

Given:

Process	Arrival	Burst
P1	0	9 ms
P2	1	5 ms
P3	3	2 ms

Practice: SRTF

0	1	3	5	8	16
P1	P2	P3	P2	P1	

Process	Completion	Arrival	TAT
P1	16	0	16
P2	8	1	7
P3	5	3	2

Process	TAT	Burst	WT
P1	16	9	7
P2	7	5	2
P3	2	2	0

Summary

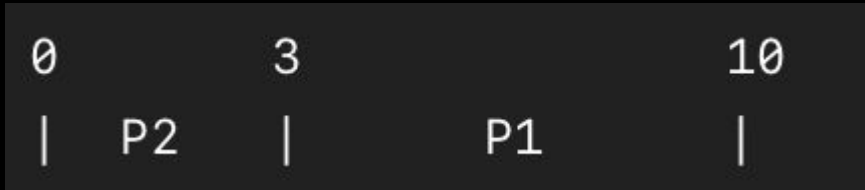
- $TAT = Completion - Arrival$
 - $WT = TAT - Burst$, Therefore:
- $WT = Completion - Arrival - Burst$

Practice: SJF

Two processes P1 and P2 arrive at time 0 with CPU burst times 7 ms and 3 ms. Using SJF (non-preemptive) scheduling, draw the execution order and compute the average turnaround time.

Practice: SJF

Two processes P1 and P2 arrive at time 0 with CPU burst times 7 ms and 3 ms. Using SJF (non-preemptive) scheduling, draw the execution order and compute the average turnaround time.



Process	Waiting Time	Turnaround Time
P1	3 ms	10 ms
P2	0 ms	3 ms

Practice: FCFS

Four processes P1, P2, P3, P4 have burst times 4, 7, 2, 5 ms and all arrive at time 0. Using FCFS scheduling algorithm (a) Compute waiting time and turnaround time for each. (b) Compute the average waiting time and average turnaround time.

Practice: FCFS

Four processes P1, P2, P3, P4 have burst times 4, 7, 2, 5 ms and all arrive at time 0. Using FCFS scheduling algorithm (a) Compute waiting time and turnaround time for each. (b) Compute the average waiting time and average turnaround time.

Process	Waiting Time	Turnaround Time
P1	0 ms	4 ms
P2	4 ms	11 ms
P3	11 ms	13 ms
P4	13 ms	18 ms

Average Waiting Time: 7ms

Average Turnaround Time: 11.5ms

Practice: SRTF

Three processes P1, P2, P3 arrive at times 0, 2, and 4 ms with CPU burst times 7, 4, and 1 ms respectively. Using preemptive SRTF scheduling: (a) Compute waiting and turnaround time for each process. (b) Compute the average turnaround time.

Practice: SRTF

Three processes P1, P2, P3 arrive at times 0, 2, and 4 ms with CPU burst times 7, 4, and 1 ms respectively. Using preemptive SRTF scheduling: (a) Compute waiting and turnaround time for each process. (b) Compute the average turnaround time.

Process	Completion	Waiting Time	Turnaround Time
P1	12 ms	5 ms	12 ms
P2	7 ms	1 ms	5 ms
P3	5 ms	0 ms	1 ms

Average Turnaround Time: 6ms

Practice Assignment

1. Three processes P1, P2, P3 arrive at time 0 with CPU burst times of 6, 2, and 4 ms respectively. Using the SJF (non-preemptive) scheduling algorithm, draw the execution order and compute the average waiting time.
2. Four processes P1, P2, P3, P4 arrive at time 0 with CPU burst times 5, 3, 8, 6 ms respectively. Using FCFS scheduling algorithm (a) Compute the waiting time and turnaround time for each process. (b) Compute the average waiting time and average turnaround time.
3. Three processes P1, P2, P3 arrive at times 0, 1, and 2 ms with CPU burst times 8, 4, and 9 ms respectively. Using preemptive SRTF (Shortest Remaining Time First) scheduling: (a) Compute the waiting time and turnaround time for each process. (b) Compute the average waiting time.

Inter Process Communication

Inter Process Communication

- Application logic in a single system is often distributed across multiple processes: why?
 - *Different processes developed independently by different teams*
 - *Different programming languages and frameworks used for different tasks*
- Processes in a system do not share any memory with each other by default, so how do they communicate information with each other?
 - *Cannot share variables or data structures in programs across processes*
- Parent and child have identical but separate memory images after fork, changes made in one process and not seen by other
- Inter-process communication (IPC) mechanisms, available via operating system syscalls, allow processes to exchange information

Inter Process Communication

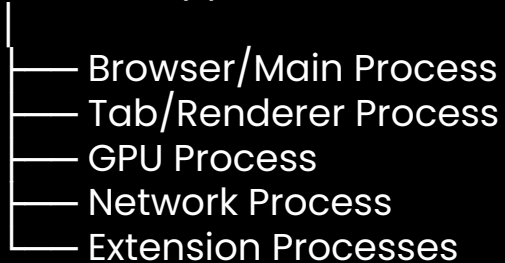
Example: web applications typically composed of multiple processes

- Web server process handles HTTP (web) requests/responses
- Written in a language like C/C++ for high performance
- Returns responses for static content directly by reading files from disk
- Requests needing dynamic response are handled by application server
- App server parses HTTP requests, generates HTTP response according to the business logic specified by user, sends response back to client via web server
- Scripting languages may be used for easy text parsing and manipulation
- Application server stores/retrieves app data in a database
- Several web application frameworks available to build web applications having such architectures, e.g., Python Django, React etc.

Inter Process Communication

For example, a web browser is one application, but it may create many processes

Browser Application



Inter Process Communication Mechanisms

There are two method of IPC,

1. shared memory and
2. message passing.
 - a. *Sockets,*
 - b. *Message Queues or*
 - c. *Pipes.*

An operating system can implement both methods of communication.

Inter Process Communication: Shared Memory

- Processes can both access same region of memory via shmget() system call
 - *int shmget (key_t key, int size, int shmflg)*
 - *By providing same key, two processes can get same segment of memory*
- Can read/write to memory to communicate
- Need to take care that one is not overwriting other's data: how?

Inter Process Communication Mechanisms

- Unix domain sockets: processes open sockets, send and receive messages to each other via socket system calls
- **Shared memory**: same physical memory frame mapped into virtual address space of multiple processes in order to share memory
- **Message queues**: sender posts a message to a mailbox, receiver retrieves message later on from mailbox
- **Pipes**: unidirectional communication channel between two processes
- **Signals**: specific messages via kill system call
- Different IPC mechanisms are useful in different scenarios

Inter Process Communication

Common IPC mechanisms

IPC mechanism

Typical system calls / APIs

Pipe

`pipe()`, `read()`, `write()`

Named pipe (FIFO)

`mkfifo()`, `open()`, `read()`, `write()`

Message queue

`msgsnd()`, `msgrcv()`

Shared memory

`shmget()`, `shmat()`

Socket

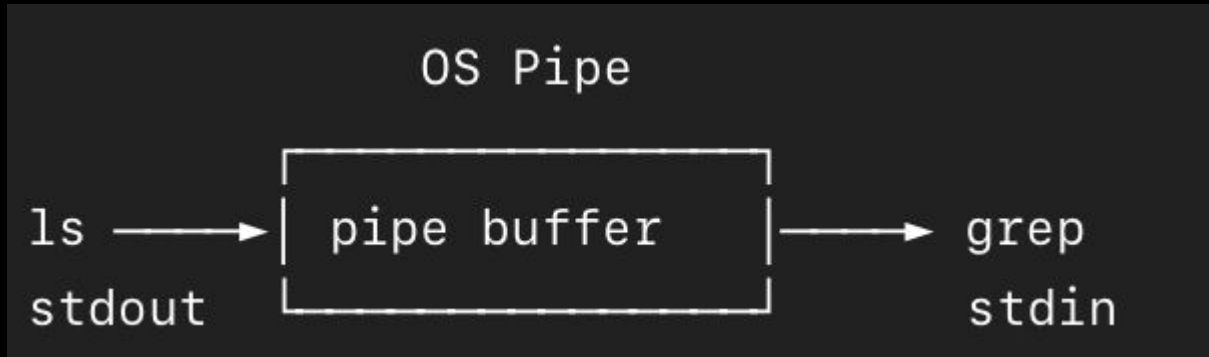
`socket()`, `bind()`, `send()`, `recv()`

Signal

`kill()`, `sigaction()`

Inter Process Communication: Pipes

- Pipe system call returns two file descriptors
 - Read handle and write handle
 - A pipe is a half-duplex communication
 - Data written in one file descriptor can be read through another
- Regular pipes: both fd are in same process (how it is useful?)
 - Parent and child share fd after fork
 - Parent uses one end and child uses other end
 - Named pipes: two endpoints of a pipe can be in different processes
- Pipe data buffered in OS buffers between write and read

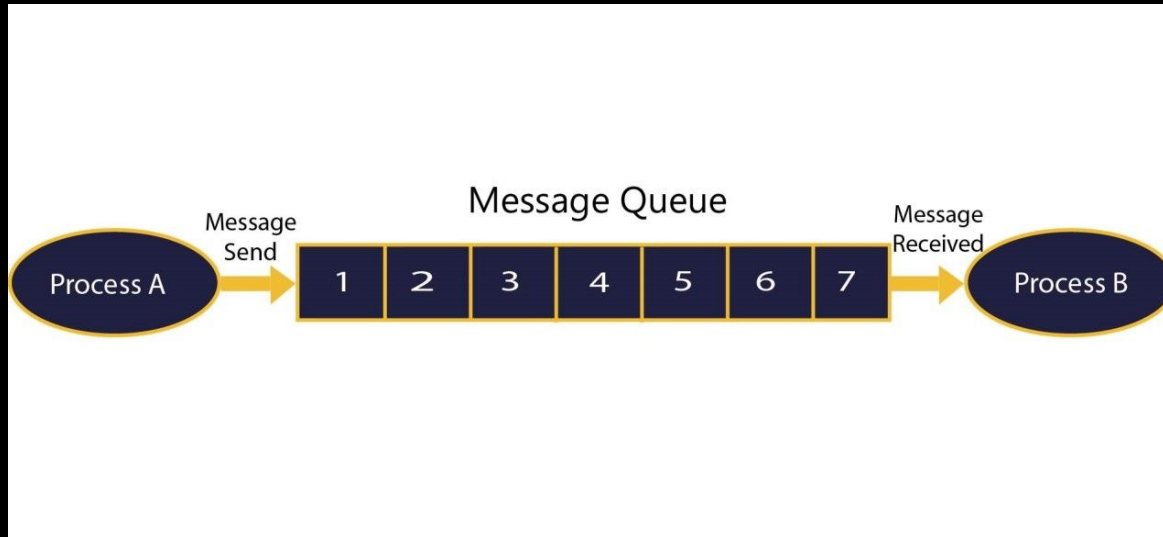


Inter Process Communication: Sockets

- A socket is an operating system abstraction that **serves as an endpoint** for sending and receiving data between two programs, either on the same device or across a computer network
 - *The socket is the **combination of IP address and software port number** used for communication between multiple processes.*
 - *For eg: 192.168.1.67:80, 155.2.12.23:77, etc.*
- Sockets can be used for two processes on same machine or different machines to communicate
 - *TCP/UDP sockets across machines*
 - *Unix sockets in local machine*
- Communicating with sockets
 - *Processes open sockets and connect them to each other*
 - *Messages written into one socket can be read from another*
- OS transfers data across socket buffers

Inter Process Communication: Message Queue

- Mailbox abstraction
- Process can open a mailbox at a specified location
- Processes can send/receive messages from mailbox
- OS buffers messages between send and receive



Inter Process Communication

Pipe = Stream (ABCDEF), Simple, Usually unidirectional,

Message Queue = Messages ([ABC] [DEF])

Socket = Network/Endpoint, Powerful

Typical use

Parent-child -> Pipe

Process coordination -> Message Queue

Client-server/network -> Socket

Blocking vs. non-blocking communication

Blocking vs. non-blocking communication

- Some IPC actions can block
 - *Reading from socket/pipe that has no data, or reading from empty message queue*
 - *Writing to a full socket/pipe/message queue*
- The system calls to read/write have versions that block or can return with an error code in case of failure
 - *A socket read can return error indicating no data to be read, instead of blocking*

Inter Process Communication: Signals

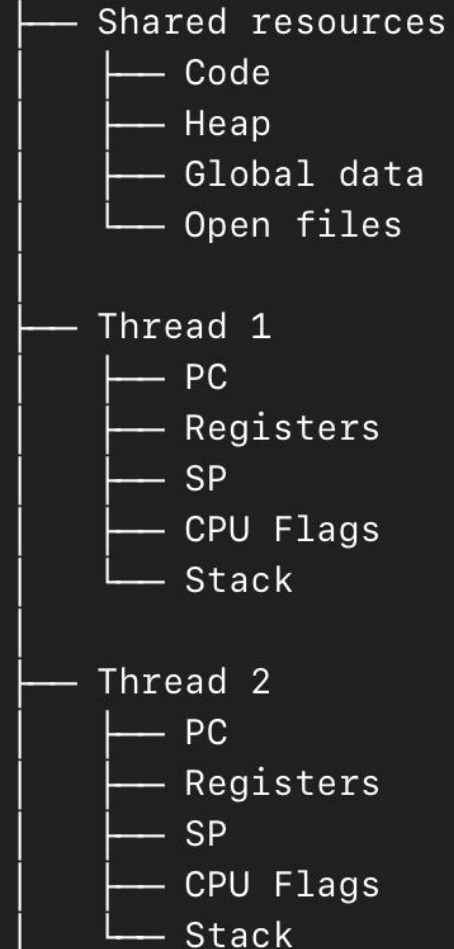
- A certain set of signals supported by OS
 - *Some signals have fixed meaning (e.g., signal to terminate process)*
- Some signals can be user-defined
 - *Signals can be sent to a process by OS or another process (e.g., if you type Ctrl+C, OS sends SIGINT signal to running process)*
- Signal handler: every process has a default code to execute for each signal
 - *Exit on terminate signal*
- Some signal handlers can be overridden to do other things

Signal Name	Description
SIGSEGV	Segmentation fault (e.g., dereferencing a null pointer)
SIGINT	Interrupt process (e.g., Ctrl-C in terminal window to kill process)
SIGCHLD	Child process has exited (e.g., a child is now a zombie after running <code>exit</code>)
SIGALRM	Notify a process a timer goes off (e.g., <code>alarm(2)</code> every 2 secs)
SIGKILL	Terminate a process (e.g., <code>kill -9 a.out</code>)
SIGBUS	Bus error occurred (e.g., a misaligned memory address to access an <code>int</code> value)
SIGSTOP	Suspend a process, move to Blocked state (e.g., Ctrl-Z)
SIGCONT	Continue a blocked process (move it to the Ready state; e.g., <code>bg</code> or <code>fg</code>)

Process vs Thread

1. Processes are usually **independent**, while threads exist as **subsets of a process**.
2. Each process has its own memory space. Threads that belong to the same process **share the same memory**.
3. A process is a **heavyweight operation**. It takes more time to create and terminate.
4. Context switching is more expensive between processes.
5. Inter-thread communication is faster for threads.

Process



Inter Process Communication: Summary

Shared Memory

⚡ Fastest (Needs Sync)

Pipes

1-way (Related Processes)

Message Queues

Flexible & Buffered

Sockets

Powerful (Across Machines)

Signals

Notifications Only

Memory Management

Memory Management

- Why we need Virtual Memory or Goals of Memory Virtualization?
 - *Security*
 - *Fragmentation*
 - *Insufficient Memory*
 - *Solutions*
- Logical Address & Virtual Address
 - *Address Translation*
 - *Memory Management Unit*
- Address Translation Mechanisms
 - *Segmentation (Obsolete, replaced by paging)*
 - *Swapping (replaced by paging)*
 - *Paging (Modern Memory Management)*
 - Demand Paging

Note: Without memory management, programs would crash into each other, overwrite data, or completely run out of space.

Memory Management: Core Problem

- In the earliest computers, a single program had the entire machine to itself.
- There was no "memory management" because there was nothing to manage - one program, one memory, no sharing.
- Modern systems run many processes concurrently

“Without memory management, every program would need to know exactly where in physical RAM it will run - and no two programs could safely coexist.”

Memory Management: Fragmentation

1 GB

200 MB
Running

300 MB
Running

250 MB
Running

274 MB

1 GB

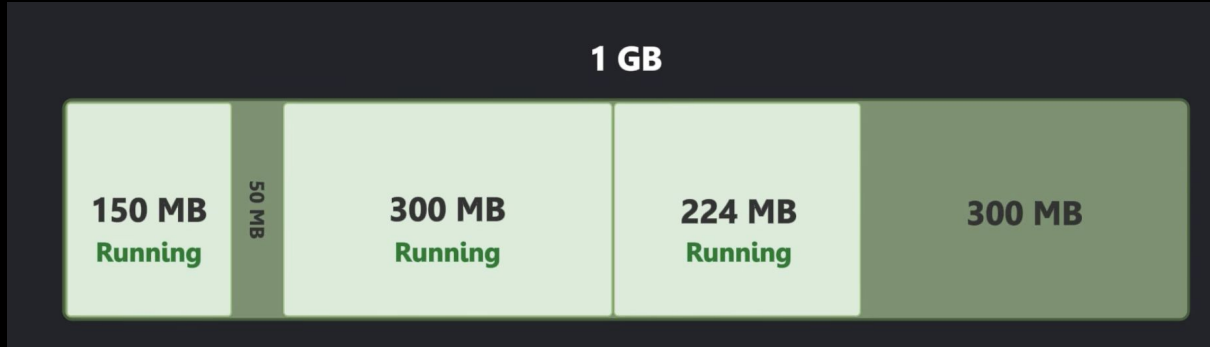
200 MB

300 MB
Running

250 MB
Running

274 MB

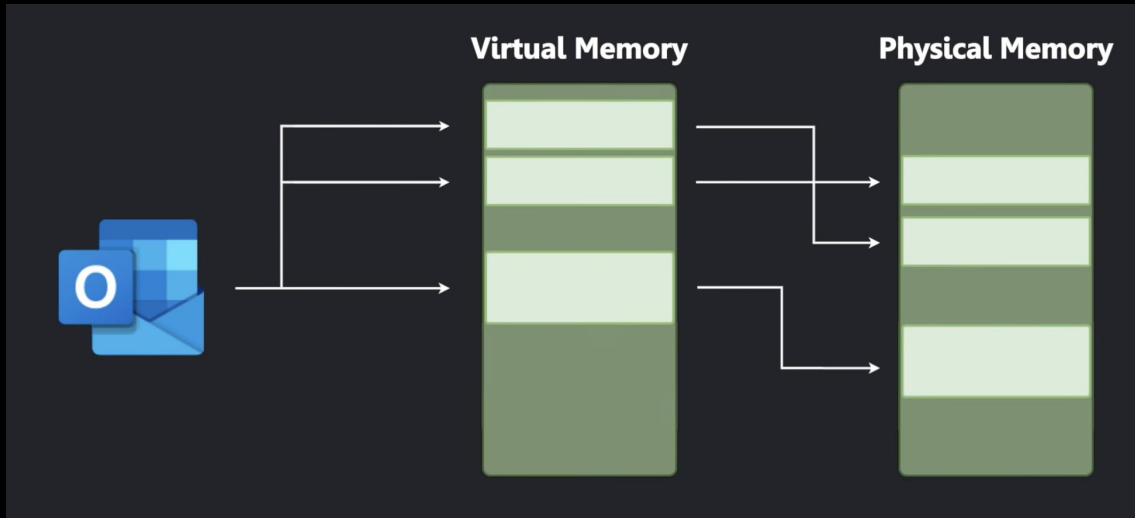
Memory Management: Fragmentation



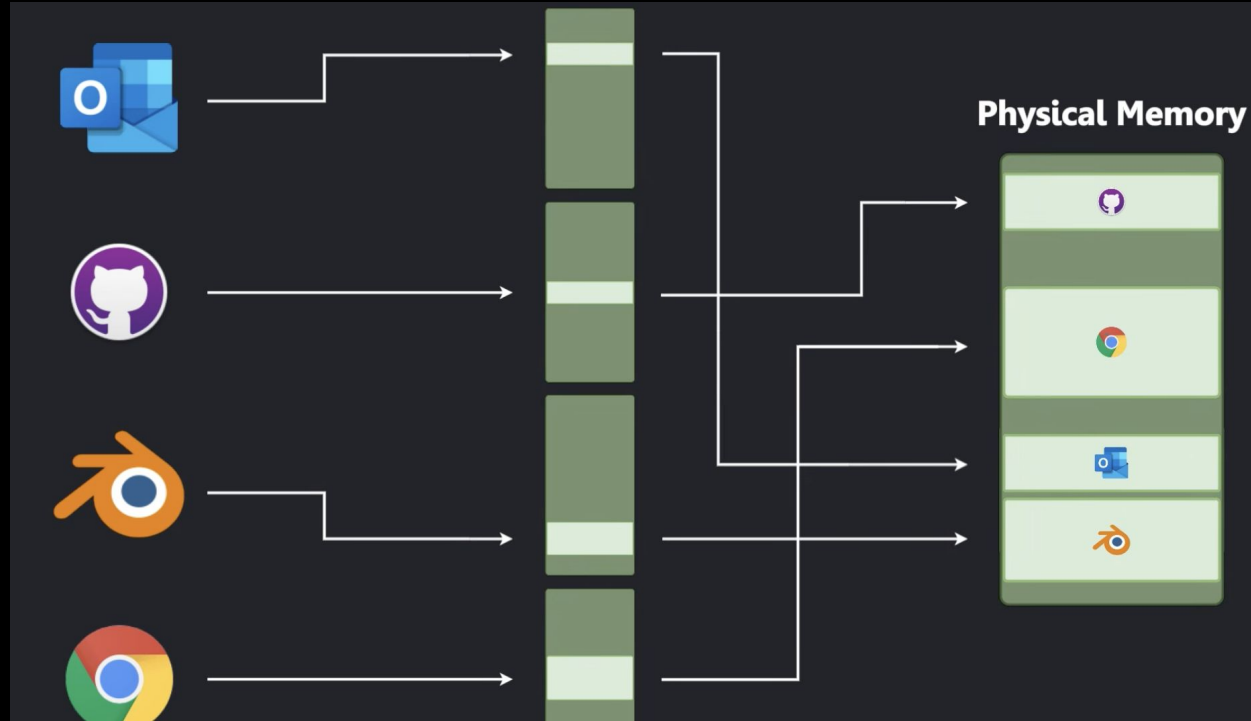
Memory Virtualization: Single Process

What is Memory Virtualization?

- A technique that **abstracts physical RAM into virtual address spaces** so each process believes it has its own large, continuous block of memory.



Memory Virtualization: Multiple Process



Memory Virtualization

- Programmers were manually splitting their programs into chunks to fit into limited core memory, and manually managing "overlays" — swapping chunks in and out by hand.
- The illusion of a large memory space, with hardware and the OS silently handling the actual movement of data between fast core memory

“Virtual memory wasn't invented for elegance — it was invented because programmers were manually doing the OS's job, splitting programs by hand to fit into tiny physical memories. Automating that illusion is where virtual memory was born.”

Memory Virtualization

"Programmers think they own all of RAM."

A programmer writes code assuming exclusive access to a large, contiguous, private address space — not assuming that all of it is simultaneously backed by physical RAM.

Virtual memory gives each process three illusions simultaneously:

- (1) that it has the entire address space to itself,
- (2) that its memory is contiguous, and
- (3) that it has more memory available than physically exists in the machine.

Memory Virtualization

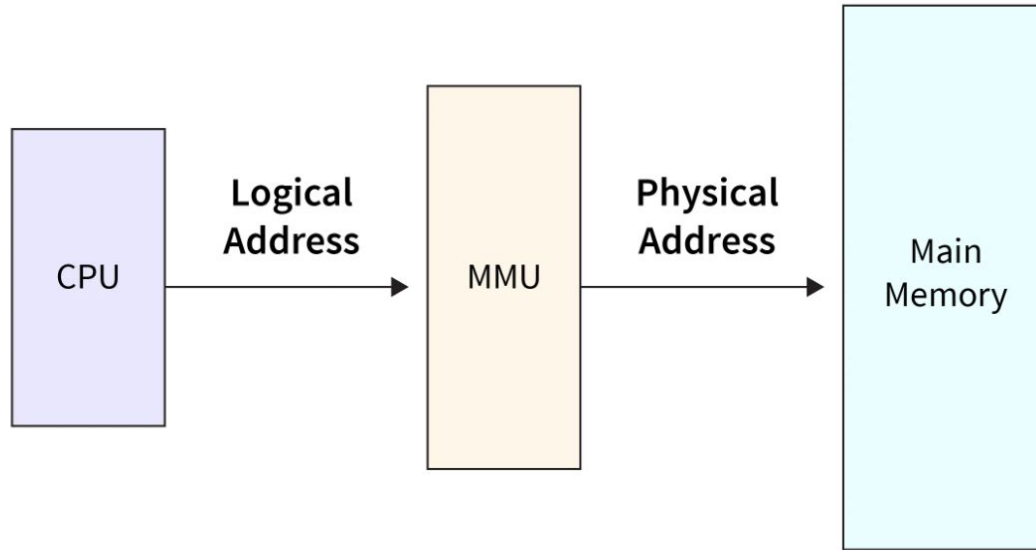
Summary: Why do we need to Virtualize Memory?

1. Avoid Manual Handling of Memory by Developers
2. Larger-Than-RAM Capacity: 8 GB RAM & 12 GB Game
3. Simplified, Contiguous Memory
4. Memory Protection and Permissions

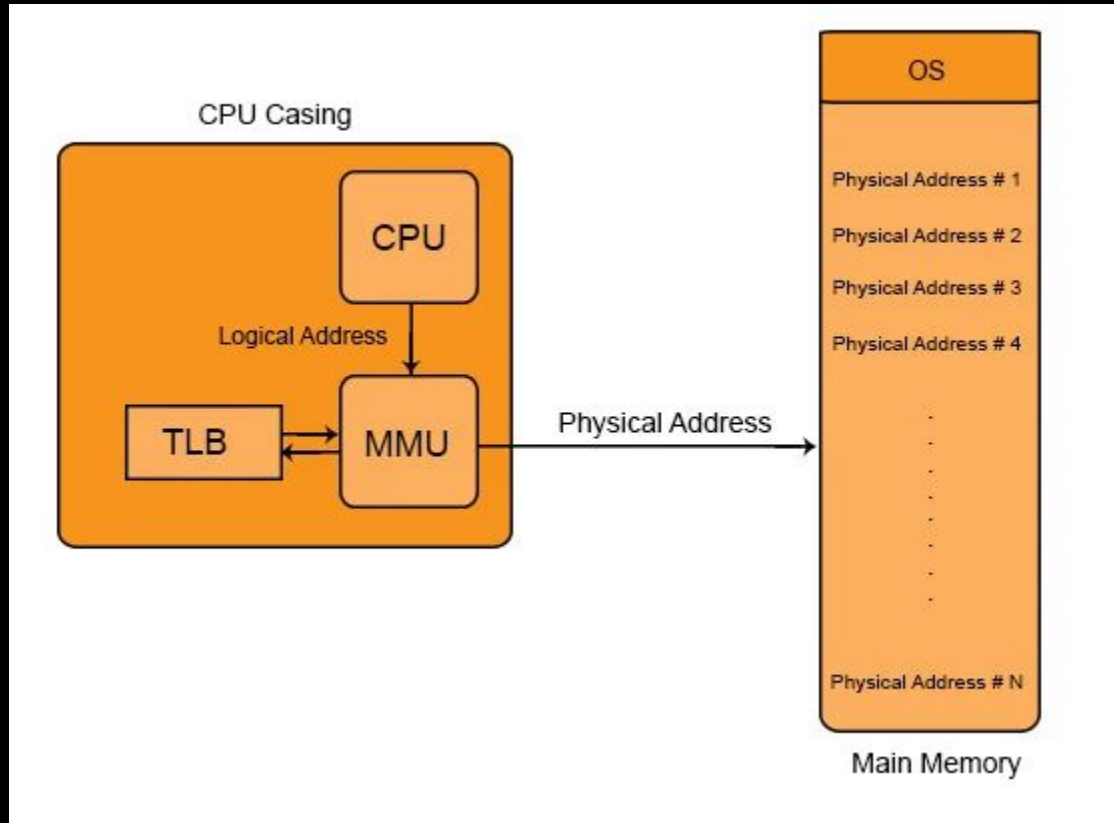
Logical and Physical Address

- In the world of computer operating systems (OS), handling memory is really important.
- One important idea in memory management is dividing memory into two types:
 - *logical and*
 - *physical address spaces.*
- Logical address
- Physical address
- Memory Manager

Logical and Physical Address



MMU



Logical Address

Logical Address (Virtual Address):

1. Generated by the CPU during program execution.
2. Represents a location in memory from the process's perspective.
3. Unique for each running process.
4. Used for accessing data and instructions.

Physical Address

Physical Address:

1. Represents the actual location in memory hardware.
2. Fixed and does not change during program execution.
3. Used for direct data retrieval from RAM or storage.

Logical and Physical Address

Memory Manager:

1. In charge of converting logical addresses to physical addresses.
2. Ensures proper isolation and memory allocation for running processes.

Multitasking:

1. Logical addresses enable multitasking by allowing multiple processes to run concurrently.
2. Each process has its own set of logical addresses.

Logical Address vs Physical Address: Differences

Difference	Logical Address	Physical Address
Definition	Generated by the CPU during program execution, representing a process's view of memory.	Represents the actual location in memory hardware, pointing directly to data in hardware memory.
Responsibility	Managed by the CPU and generated for each running process, ensuring process isolation.	Managed by the hardware and remains static during program execution, providing stable memory addressing.
Translation	Requires translation by the memory manager or hardware before accessing data; it's a virtual address.	Does not require translation; it directly represents the data's physical location in memory.
Uniqueness	Unique for each running process, allowing processes to run independently.	Unique for each storage cell in memory, providing precise addressing within hardware memory.

Logical and Physical Address

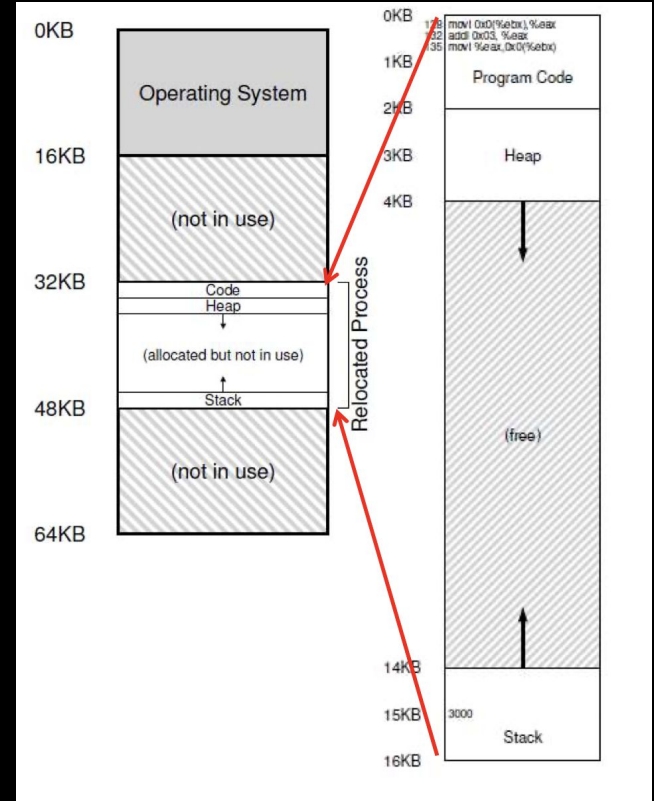
Walk through the sequence explicitly:

1. CPU fetches an instruction, which references logical address 346.
2. This logical address is sent to the MMU — the CPU itself never touches physical memory directly.
3. The MMU applies a translation rule (in this session: base + limit) to compute the physical address.
4. If the address is invalid (out of bounds), the MMU raises a hardware trap, and the OS handles it (typically terminating the process with a segmentation fault).
5. If valid, the physical address goes out on the memory bus and the data/instruction comes back.

How are virtual address spaces mapped to physical memory?

Simplest form of memory management: **base and bound**

- Place memory image [0,N] contiguously starting at memory address base B
- Virtual address X translated to physical address $B+X$
- Access to virtual addresses beyond N will not be permitted
- OS provides base and bound to MMU for translation/error checking
- When CPU access a VA, MMU computes $PA = VA + \text{base}$, physical memory accessed with PA



Role of OS vs MMU

- OS allocates memory, builds translation information of process
 - *But OS does not do the actual address translation on every memory access*
 - *Why? Once user code starts running on CPU, OS is out of the picture (until a trap)*
 - *When process is switched in, translation information is provided to MMU*
- CPU runs process code, accesses code/data at virtual addresses
 - *Virtual addresses translated to physical addresses by MMU*
 - *Actual physical memory is accessed using physical addresses*
- MMU raises a trap if there is any error in the address translation
 - *CPU executes trap instruction, OS code runs to handle the error*
- OS gives new information to MMU on every context switch

The CPU understands Virtual Memory Addresses.

The OS allocates Physical Memory Addresses.

PYQs

Which of the following is the primary purpose of virtual memory?

- A. Increase CPU clock speed
- B. Allow execution of processes larger than physical memory
- C. Increase cache size
- D. Eliminate page faults

PYQ

Which of the following is the primary purpose of virtual memory?

- A. Increase CPU clock speed
- B. Allow execution of processes larger than physical memory
- C. Increase cache size
- D. Eliminate page faults

Answer:

- B. Allow execution of processes larger than physical memory

Virtual memory gives a process the abstraction of a large address space even when physical RAM is smaller.

Practice

A1. A process has base = 2400 and bound = 670. It issues virtual address 410. What is the physical address, and is the access valid?

1. 2810; valid
2. 2810; invalid
3. 1990; valid
4. 3070; valid

Practice

A1. A process has base = 2400 and bound = 670. It issues virtual address 410. What is the physical address, and is the access valid?

1. 2810; valid
2. 2810; invalid
3. 1990; valid
4. 3070; valid

1. $410 < 670$, so valid; $PA = 2400 + 410 = 2810$. Option 3 subtracts instead of adding; option 4 adds the bound instead of the VA.

Practice

A2. A process has base = 5600 and bound = 940. It issues virtual address 1260. What happens?

1. Physical address 6860 is accessed
2. Physical address 4340 is accessed
3. The MMU raises a trap and the OS handles the fault
4. The address is clipped to 940, so physical address 6540 is accessed

Practice

A2. A process has base = 5600 and bound = 940. It issues virtual address 1260. What happens?

1. Physical address 6860 is accessed
 2. Physical address 4340 is accessed
 3. The MMU raises a trap and the OS handles the fault
 4. The address is clipped to 940, so physical address 6540 is accessed
3. $1260 > 940$, so the MMU traps before any memory is touched. Option 1 is the classic mistake of adding without checking the bound.

Practice

A3. When the OS switches the CPU from process P to process Q, who sets the base and bound registers to Q's values?

1. Process Q itself , as the first instruction it runs in user mode
2. The OS, in kernel mode, using the values saved for Q
3. The MMU, which computes them from Q's PID automatically
4. Nobody; all processes share one base and bound

Practice

A3. When the OS switches the CPU from process P to process Q, who sets the base and bound registers to Q's values?

1. Process Q itself , as the first instruction it runs in user mode
 2. The OS, in kernel mode, using the values saved for Q
 3. The MMU, which computes them from Q's PID automatically
 4. Nobody; all processes share one base and bound
2. Only the kernel can load base and bound. It takes Q's values from Q's saved state (PCB) during the context switch.

Practice

A4. Why are the instructions that load the base and bound registers privileged (kernel mode only)?

1. Because they are slower than ordinary instructions
2. Because the compiler must insert them at build time
3. Because user programs never need to know their own addresses
4. Because a user process could otherwise change its own limits and reach other processes' memory

Practice

A4. Why are the instructions that load the base and bound registers privileged (kernel mode only)?

1. Because they are slower than ordinary instructions
 2. Because the compiler must insert them at build time
 3. Because user programs never need to know their own addresses
 4. Because a user process could otherwise change its own limits and reach other processes' memory
4. If a user process could load these registers, it could raise its own bound or move its base and read or overwrite any memory, so protection would be meaningless.

Practice

1. Validity table. A process has base = 1400 and bound = 360. For each virtual address, say whether it is valid and give the physical address if it is: (a) 75 (b) 510 (c) 290 (d) 930.
2. B2. Working backwards. For the same process (base = 1400, bound = 360), which virtual address, if any, produced each physical address? (a) 1630 (b) 1955. Justify each answer.

Solutions

1. Solution:

- a. $75 < 360$ Valid, $PA = 1400 + 75 = 1475$
- b. $510 > 360$ Invalid, MMU traps
- c. $290 < 360$ Valid, $PA = 1400 + 290 = 1690$
- d. $930 > 360$ Invalid, MMU traps

2. Solution:

- a. $VA = 1630 - 1400 = 230$, and $230 < 360$, so VA 230 produced it.
- b. $1955 - 1400 = 555$, but $555 > 360$. No valid VA of this process maps to 1955: that physical address is outside the process's region, so the process can never reach it.

Swapping

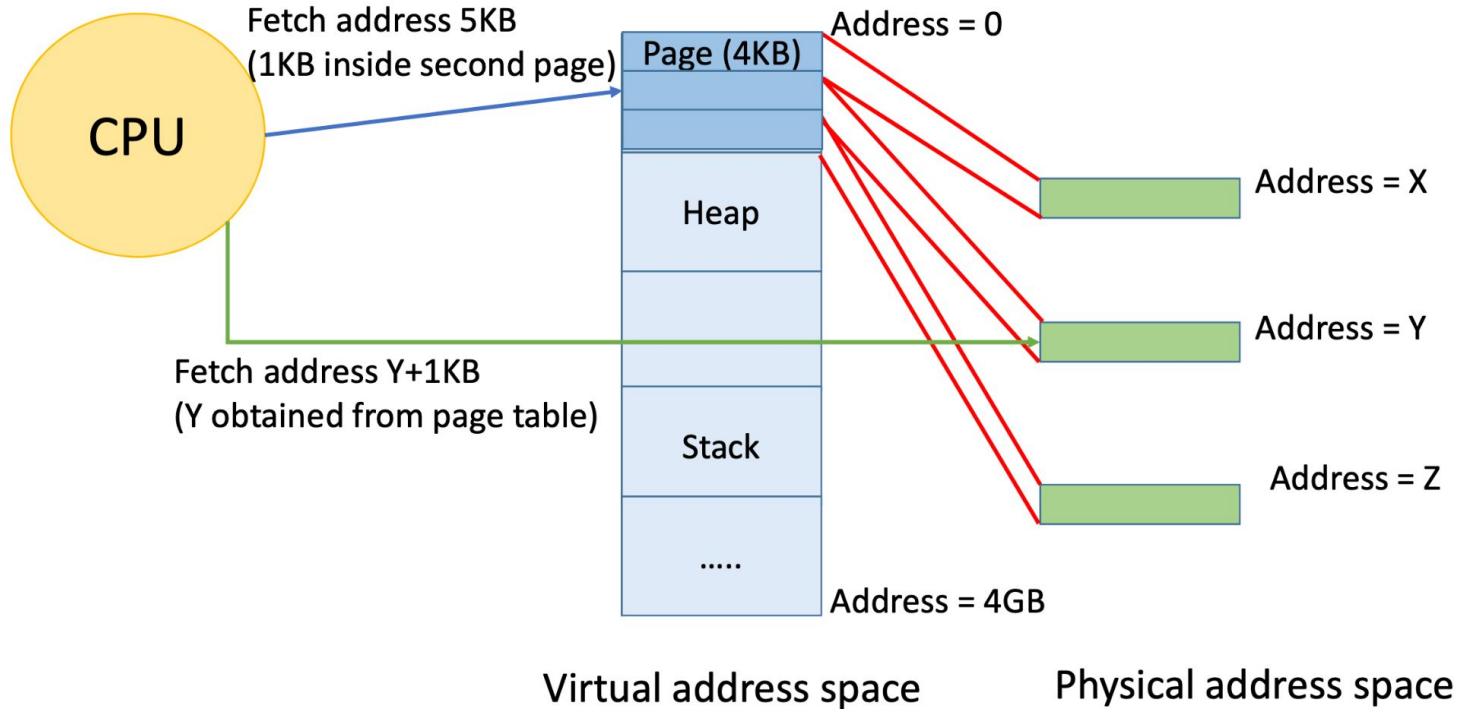
- In classic operating systems, swapping meant moving an entire process (all of its code, stack, data, and heap) out of RAM onto a hard drive to free space
- Modern operating systems no longer swap whole processes.
- Instead, they use a hybrid approach often called paging out (or page swapping). They only kick individual, inactive pages out to the storage drive (into a pagefile or swap space) while keeping the active pieces of the process running in RAM.

Paging

Paging

- Widely used memory management system today
- Virtual address space divided into **fixed size pages**
- Each page is assigned a free physical frame by OS
- Memory allocation is at granularity of fixed size pages (e.g., 4KB)
- Why paging? Avoids **external fragmentation**
 - *No wastage of space due to gaps between allocated and free memory*
 - *Internal fragmentation may be there (space wasted inside partially filled page)*
- Disadvantage: **internal fragmentation** (partially filled pages)
- Page table of a process maps **virtual page numbers (VPN)** to **physical frame numbers (PFN)** for every page

Paging

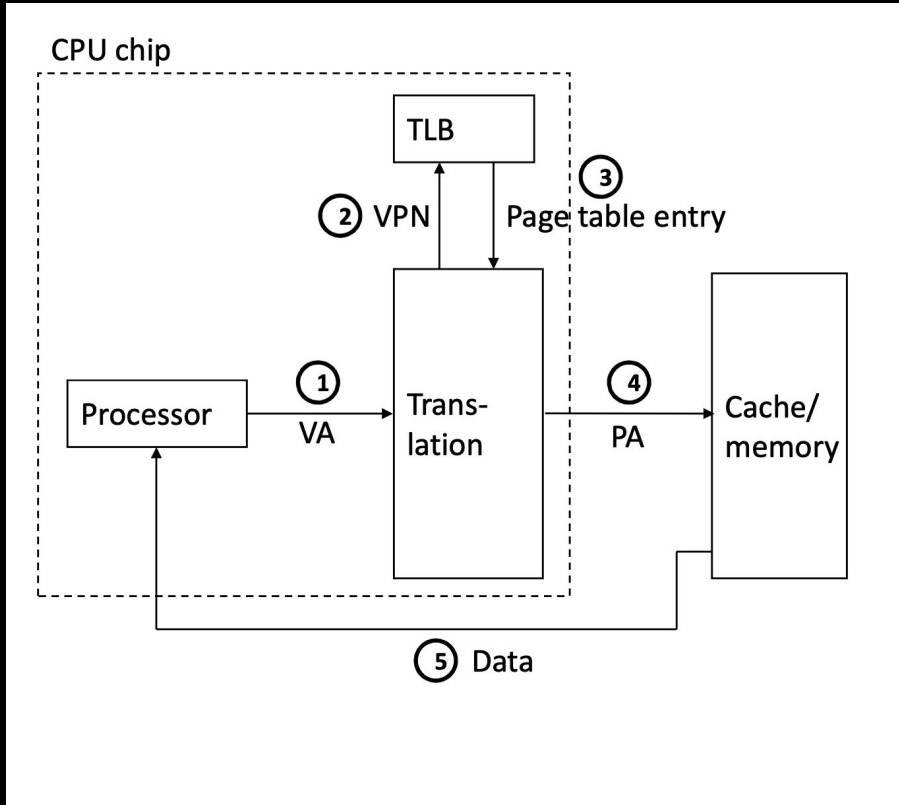


Translation Lookaside Buffer (TLB)

- Overhead of memory translation: every memory access preceded by extra memory accesses to read page table
- To reduce this overhead, MMU caches the most recent translations in translation lookaside buffer (TLB)
- TLB only caches page table entries (VPN PFN mappings), not actual memory contents
 - *Different from CPU caches that cache actual memory contents*
- If TLB hit, fetch memory contents in one memory access
- If TLB miss, MMU must perform extra memory access for page table access (“page table walk”)
- TLB flush on context switch: mappings cached in TLB change

TLB hit

1. CPU accesses virtual address
2. MMU looks up page number in TLB
3. If TLB hit, page table entry is available, physical address computed
4. CPU directly accesses required code/data using physical address



TLB miss

1. CPU accesses virtual address
2. MMU looks up page number in TLB, cannot find entry
3. MMU looks up page table in memory to find page table entry
4. Page table entry populated in TLB for future use
5. MMU computes physical address using which CPU accesses main memory

